

School of Electronic Engineering
and Computer Science

Final Report

Programme of study:

BEng Electrical and Electronic
Engineering

Project Title:

**Gesture Recognition
Using Machine
Learning and Self-
Calibrating Techniques
for Remote Control
Cars.**

Supervisor:

Dr Luk Arnaut

Student Name:

Raakin Ali

Final Year
Undergraduate Project 2024/25



Date: 06/05/2025

Abstract

The interpretation of physical gestures for controlling devices is known as gesture recognition. One of the emerging applications of this technology is in the domain of remote-controlled vehicles, where traditional input methods can be replaced with more intuitive hand gestures. However, achieving precise and responsive control in real-time presents several challenges.

This report describes a hand-based gesture controller for a remote-control car, using neural networks and calibration techniques to ensure low latency and high accuracy. A solution that enables users to control the car's movements through simple hand gestures, making the experience more engaging and accessible. It details the process of gathering requirements, designing the gesture recognition model, integrating it with the car's control system, and evaluating whether the final implementation meets the desired performance, accuracy, and real-time responsiveness objectives.

C contents

Table of Contents

Chapter 1: Introduction.....	6
1.1 Background.....	6
1.2 Problem Statement	6
1.3 Aim.....	7
1.4 Objectives	7
1.5 Research Questions	7
Chapter 2: Literature Review.....	8
2.1 Gesture Recognition Technologies	8
2.1.1 Vision-Based Gesture Recognition (VBGR).....	8
2.1.2 Sensor-Based Gesture Recognition (SBGR)	9
2.1.3 Most suitable Gesture recognition method for Remote car implementation .	10
2.2 Machine Learning in Gesture Recognition	11
2.2.1 Overview of Machine Learning in Gesture Recognition	11
2.2.2 Vision Transformer Technique [7]	11
2.2.3 Convolutional Neural Networks (CNNs) [8].....	12
2.2.4 Support Vector Machines (SVM) [9].....	12
2.2.5 Benefits and Challenges of Machine Learning in Gesture Recognition.....	12
2.3 Auto-Calibration Techniques	13
2.3.1 Min-Max Normalisation [11].....	14
2.3.2 Zeroing and Drift Correction [12]	14
2.4 Comparison of Similar RC gesture controllers	15
2.5 Proposed Solution	16
Chapter 3: Design And Method	17
3.1 System Architecture.....	17
3.2 Hardware Design	19

3.2.1	RC Car design	19
3.2.2	Gesture Controller design.	23
3.3	Gesture Recognition Model Design	26
3.3.1	Data Collection	26
3.3.2	Data Preprocessing (Using Edge Impulse Software)	28
3.3.3	Training the neural network model	31
Chapter 4:	Implementation	36
4.1	Languages	36
4.1.1	C/C++	36
4.1.2	Python	36
4.2	Essential Libraries	36
4.2.1	Gestures_inferencing.h	36
4.2.2	Arduino.h	36
4.2.3	Wire.h	36
4.2.4	Adafruit_BNO08x.h	37
4.2.5	esp_now.h	37
4.2.6	WiFi.h	37
4.3	Development of Core Functionalities.....	38
4.3.1	Closed-loop Mean Bias Calibration (Appendix B-C)	38
4.3.2	Neural network implementation (E-G)	39
4.3.3	Remote Control Car Code (Appendix G)	39
4.3.4	Speed evaluation code (Appendix H).....	39
Chapter 5:	Testing And Results.....	40
5.1	Gesture Controller Performance Testing	40
5.2	Auto Calibration System Testing	42
5.2.1	Sensor Drift Test	42
5.2.2	Calibrating System - Error Correction response	44
Chapter 6:	Evaluation.....	45
6.1	Performance Assessment.....	45
6.2	Auto Calibration Test Anyalsis	46

6.3	Comparisons to Existing Systems	46
Chapter 7: Conclusion.....		47
7.1	Achievements	47
7.2	Challenges	47
7.3	Future Improvements.....	48
7.4	Final Conculsions	48
References		49
Appendix A – Source code		51
Appendix B - Initial Mean Bias Calibration		52
Appendix C - Rapid Mean Bias Calibration		53
Appendix D - Gesture classification evaluation		54
Appendix E - RC Car Direction gesture classification.....		55
Appendix F - Non-car directional gestures classification		56
Appendix G – Remote Control Car Code		57
Appendix H - Speed evaluation code		62
Appendix I – BNO08X Circuit Diagram [17].....		63
Abbreviations list.....		64

Chapter 1: Introduction

1.1 Background

Gesture recognition technology has rapidly emerged as a key enabler of intuitive and natural human-computer interaction (HCI)[1]. It allows users to control devices and systems through hand or body movements. Gesture control systems are transforming the way humans interact with technology. These systems have found applications across a diverse range of fields, including gaming, robotics, virtual reality, smart home automation, and assistive technologies. By dropping the need for physical controllers, gesture recognition systems provide a seamless and often more accessible mode of interaction, facilitating more natural and immersive user experiences.

Despite the growing popularity of gesture recognition systems, they face several challenges that hinder their effectiveness in real-world applications. Many traditional systems rely on pre-defined gestures, which limit their adaptability to different users or environments. For instance, variations in factors such as hand size, motion speed, and lighting conditions can drastically affect the system's ability to accurately recognise gestures. Moreover, the physical placement of sensors can introduce significant errors, further compromising system performance. Additionally, manual calibration processes are often needed to optimise system accuracy, a step that can be time-consuming and reduce overall usability.

Machine learning can be implemented in gesture recognition by using algorithms like Convolutional Neural Networks (CNNs) for visual pattern classification, and Support Vector Machines (SVMs) for robust classification. These techniques enable the system to accurately recognise and adapt to complex, dynamic gestures, improving performance and user interaction. [2].

This project utilises Machine learning and sensor calibration techniques to successfully identify static gestures (These are gestures where the hand remains in a fixed position without significant movement, e.g. Thumbs up) and dynamic gestures (These involve movement and changes in position over time, e.g. a wave) with high accuracy and minimal latency.

1.2 Problem Statement

Traditional gesture controllers often face challenges in accurately interpreting gestures across different users, environments and contexts. Manual calibration processes can be time-consuming and may not adapt well to changes, limiting the usability and efficiency of these systems and often leading to latency within the system. This project looks to overcome these limitations by integrating auto-calibration and machine learning techniques to enhance recognition accuracy and adaptability.

1.3 Aim

The aim of this project is to develop an intelligent gesture control system that uses machine learning and auto-calibration to provide enhanced recognition accuracy and user adaptability for use in remote control cars.

1.4 Objectives

- Design and develop a gesture-controlled RC car as a test platform for the proposed gesture recognition system.
- Implement an auto-calibration mechanism in the gesture control system to adapt to varying user gestures and environments.
- Integrate machine learning models to improve gesture recognition accuracy and enable system adaptability over time.
- Evaluate the performance of the gesture-controlled RC car in terms of response time, accuracy, and reliability in real-world scenarios.
- Compare the proposed system's efficiency with conventional gesture-controlled remote-controlled systems to prove its effectiveness.

1.5 Research Questions

- How can auto-calibration be implemented effectively in a gesture recognition system?
- What machine learning models are most suitable for improving gesture recognition accuracy?
- How does the proposed system perform compared to traditional gesture controllers in terms of adaptability, accuracy, and user experience?
- What are the limitations of combining auto-calibration with machine learning in gesture control systems?

Chapter 2: Literature Review

2.1 Gesture Recognition Technologies

Gesture recognition technology enables human-computer interaction by interpreting human gestures through mathematical and computational models. This section provides an overview of the methods and hardware used in gesture recognition systems.

2.1.1 Vision-Based Gesture Recognition (VBGR)

Vision-based gesture recognition is a technology that allows devices to interpret human gestures and respond to them. It's a subdiscipline of computer vision that uses mathematical algorithms to analyse video images of human gestures and translate them into actions or commands [3]. Vision-based gesture recognition typically depends on three stages: gesture detection and pre-processing; gesture representation and feature extraction; gesture representation and feature extraction [4].

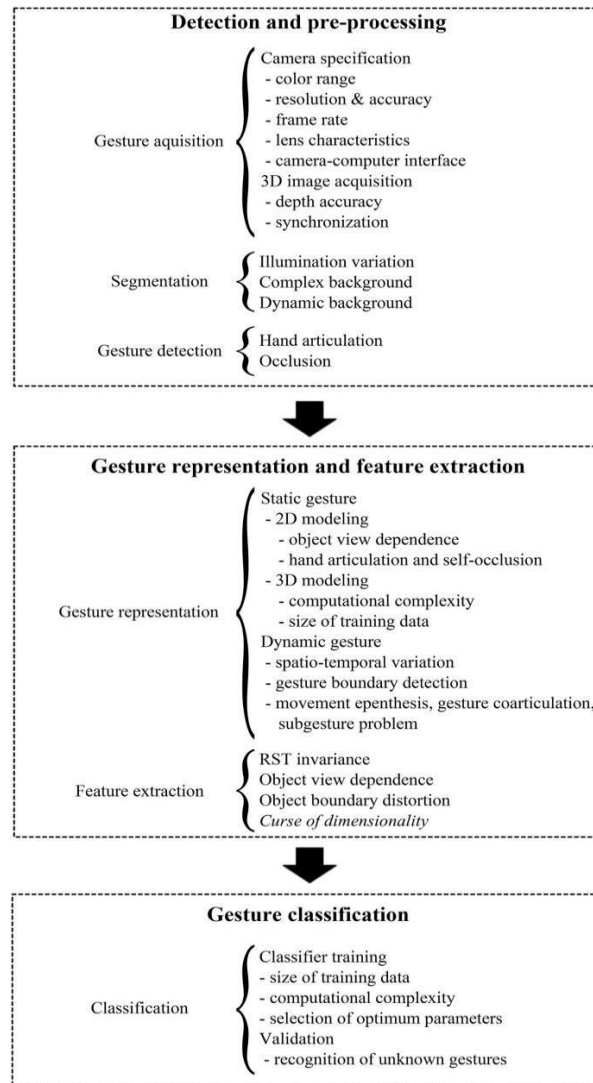


Figure 1. Distinct stages of a VGR system [4]

Detection and preprocessing:

Consists of capturing gestures using imaging devices such as cameras. Preprocessing, segments the body parts from the camera recording; accuracy is dependent on the algorithms and the quality of the camera used (Camera parameters shown in Figure 1).

Gesture representation and feature extraction:

The gesture must then be represented in a mathematical description or model to be processed. The modelling is dependent on the intended use. In some cases, only a few gestures are suitable for operations. However, if the purpose is broader and requires a larger variety of gestures. This requires the gesture to be classified (a) shape, (e.g. geometric features or non-geometric features), (b) texture or pixel value, (c) 3d model-based features, and (d) spatial features (e.g. position and motion). The gesture must be classified as a static or dynamic gesture (Parameters shown in Figure 1) [4].

Recognition:

The final stage of a vision-based gesture recognition system is classification. A classifier algorithm recognises the recorded and processed parameters and groups them into their predefined classes (supervised) or by their similarity (unsupervised).

Accurate detection of gestures is difficult because of constraints of lighting, shadows, complex background, and other factors. The complex articulated shape of the hand makes it harder to model the appearance of both static and dynamic gestures. Variation of gesture parameters due to variance in hand postures makes the recognition process more difficult. [4] VBGR is also expensive to implement as it requires sufficient computational power to process 2D and 3D imaging and High-resolution Cameras for accurate picture capture.

2.1.2 Sensor-Based Gesture Recognition (SBGR)

Sensor-Based Gesture Recognition consists of a wearable hand gesture system which uses a cluster of sensors (Inertial measurement units and Force myography) to accurately capture gestures. The Gesture signals are then analysed (Figure 2). Once the signal is identified, control signals are sent to the Controlled device.

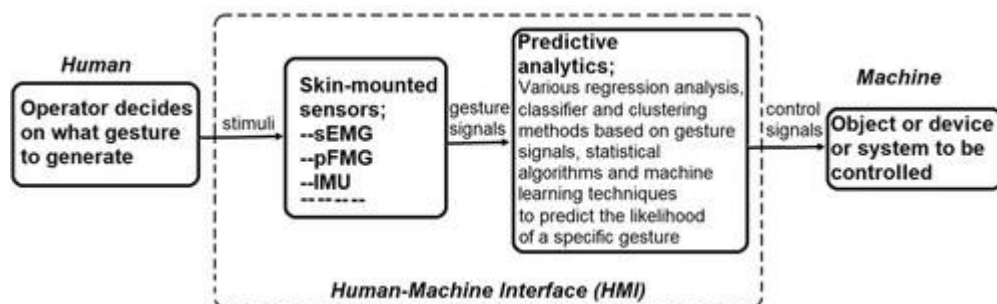


Figure 2. Distinct stages of a VGR system [5]

Mechanical sensing can be categorised into three types: force myography (FMG), inertial measurement units and flex sensor sensing.

Force myography (FMG) is a method used to monitor the mechanical signals generated during muscle contractions. This approach captures variations in muscle tension and deformation by utilising devices such as pressure sensors or strain gauges positioned on the skin. FMG measures physical forces, making it a compelling alternative for gesture recognition and other muscle-based applications.

Commonly used FMG sensors include capacitive array sensors. They can accurately detect complex gestures with an accuracy of 99% [5] in ideal conditions with minimal training. However, Capacitive sensor arrays require regular calibration to maintain accuracy, especially in dynamic environments where external factors like dust or dirt accumulation can affect the sensor's performance. Capacitor sensors also have high spatial resolution/real-time tracking abilities, which consume substantial amounts of power. This can be a challenge in battery-operated devices.

(Inertial measurement units) IMUs, which usually consist of 3-axis accelerometers and 3-axis gyroscopes, are used to record the arm's overall movement, such as the compensatory balancing movements, limb orientation and even information related to fatigue, smoothness, and the velocity of an arm's movements. However, IMUs are rarely used on their own for hand and wrist gesture recognition, as the kinematic and orientation information alone is not sufficient to detect patterns of gestures.

IMUs have a performance accuracy of 77.2–98.5% [5] in gesture recognition applications. The apparent range for performance is due to the sensor drift and calibration. When used for prolonged periods, IMUs accumulate small errors in the readings, which can cause the system to deviate from the actual position. To prevent this issue, the IMU must be regularly calibrated.

Simple gestures like swipes are easier for IMUs to track because they involve straightforward and linear movements. In contrast, more intricate or rapid gestures, such as circular motions or multi-directional movements, can create noise or cause confusion in the sensor data, leading to decreased accuracy. The speed and complexity of the gesture significantly influence how accurately an IMU can detect it.

Flex sensor, Flex sensors are also commonly used as strain sensors to measure the angle of bending. These sensors can be attached to each finger to recognise hand and finger gestures. They are usually embedded into a glove and work in conjunction with IMUs. However, they are sensitive; therefore, slight adjustments could lead to an unwanted change in resistance. This can be controlled through calibration and adding an activation zone (A voltage threshold)

2.1.3 Most suitable Gesture recognition method for Remote car implementation

For Remote control (RC) cars, vision-based recognition may not be suitable as a camera has to have a clear view of the hand. It would be impractical to have a camera follow or be attached at a certain elevation that has clear visibility of the hand in outdoor scenarios. SBGR are directly attached to the person in the form of a glove, which is more suitable for an RC car as the person can follow the car and not have to worry about sensors losing focus of the hand. Capacitive array sensors are more accurate at gesture capture, however, they are constrained by the power requirements. As this is a handheld device, large amounts of power will not be available. Therefore, IMUs and flex sensors would be a better approach as they can achieve almost equivalent gesture capture accuracy, if high-accuracy IMUs are used, with significantly less required power.

2.2 Machine Learning in Gesture Recognition

2.2.1 Overview of Machine Learning in Gesture Recognition

Machine learning (ML) is pivotal in sensor-based gesture recognition, facilitating the efficient analysis and interpretation of data obtained from various sensors, including accelerometers, gyroscopes, magnetometers and pressure sensors. These sensors record physical movements and gestures as signals, which ML algorithms process to identify and classify distinct gestures accurately.

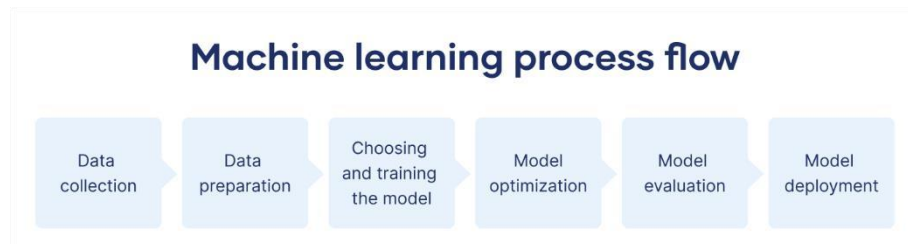


Figure 3. Machine learning process flow [6]

Data collection/ Preparation Machine learning uses Data sets to train models. For gesture-based controllers' collection of sensor data, it is necessary. The user can tabulate specific gesture paths with multiple instances of the gesture, as the user will not be able to recreate the exact movements made for the gesture. Therefore, multiple instances of the gesture can remove uncertainty with recognition. Training the model is critical for gesture-based recognition, as Machine learning models must rapidly detect and differentiate advanced gestures with minimal delay for the model to be utilised effectively.

2.2.2 Vision Transformer Technique [7]

A type of neural network that is designed to process visual data using self-attention mechanisms instead of traditional convolutional layers. They divide an image into patches and analyse their spatial relationships globally, making them highly effective for recognising complex patterns in gesture recognition. This architecture enables Vision Transformers to learn meaningful representations without relying on handcrafted convolutional filters.

Why: Traditional Convolutional Neural Networks focus on local feature extraction, which can limit their ability to capture long-range dependencies in images. Vision Transformers address this limitation by using self-attention to model relationships between different regions of an image, allowing them to recognise intricate hand gestures more effectively. Their ability to process entire gesture frames holistically makes them particularly suitable for vision-based gesture recognition.

Best Use Cases: Ideal for recognising static and dynamic gestures in real-time applications such as sign language translation, virtual reality interactions, and autonomous systems. Vision Transformers excel in tasks where global context is crucial, such as distinguishing between similar hand shapes or understanding gesture sequences. **Drawback:** Vision Transformers require significantly more computational resources and large datasets for effective training. Their high processing cost makes them less suitable for microcontroller devices with limited hardware capabilities.

2.2.3 Convolutional Neural Networks (CNNs) [8]

A class of deep learning models which is typically used to process data with grid-like structures, e.g. Image or time series data. It uses convolution layers to extract partial or temporal patterns such as edges, shapes or motion. CNNs are best used for image recognition, object detection and pattern analysis. The ability to detect patterns is the main reason it could be implemented into sensor-based gesture control models.

Why: Sensor data can be represented as structured formats like spectrograms or heatmaps, where CNNs excel at identifying spatial patterns. CNNs are effective for recognising gestures involving multiple sensors, where interactions between features need to be modelled.

Best Use Cases: Applications where sensor data can be transformed into 2D/3D representations or when working with multi-sensor fusion. Suitable for gaming, AR/VR, or multi-modal systems. **Drawback:** Requires preprocessing to structure data into a compatible format.

2.2.4 Support Vector Machines (SVM) [9]

A powerful machine learning algorithm used for linear and non-linear classification also regression, and anomaly detection. SVMs demonstrate remarkable versatility, making them ideal for a wide range of applications, including categorising text, recognising images, detecting spam, identifying handwriting, analysing expression, detecting faces, and identifying anomalies. It can also be used for pattern recognition for gesture recognition.

Why: A reliable conventional machine learning technique for gesture detection in small to medium-sized, well-labelled datasets. Performs well on data with clear boundaries between gesture classes after feature extraction.

Best Use Cases: Lightweight systems or embedded devices with limited computational power. Simple gestures or applications where real-time inference is critical (e.g., wearable devices). **Drawback:** Not as scalable as deep learning methods for large datasets or overly complex gestures.

2.2.5 Benefits and Challenges of Machine Learning in Gesture Recognition

Machine learning and deep learning are powerful tools that can be implemented in gesture controllers to allow for greater responsiveness in systems, The main benefit is that they allow for the processing and implementation of more complex gestures through vigorous training. Without the use of Machine learning and Deep learning models, the gesture controls will lack the ability to accurately distinguish gestures.

A key challenge of implementing ML in gesture recognition systems is computational power. Gesture recognition lacks high processing power as they typically use microcontrollers, however, Support vector machine (SVM) can combat the issue as they are less resource-intensive. Another major challenge is calibration and sensor drift. If the system isn't regularly calibrated, the machine learning model may not operate as intended, as this is caused by the small drift in values which can occur over time. Sensor drift can be removed through regular calibration. The next section describes techniques that can be implemented to reduce this issue.

2.3 Auto-Calibration Techniques

Inertial Measurement Units (IMUs) are pivotal in numerous modern applications, from drones to autonomous vehicles. They heavily rely on a process called intrinsic calibration, which corrects internal errors and biases for accurate measurements [10].

Principle of Operation of IMUs

IMUs operate by combining data from multiple sensors to measure motion and orientation. They typically consist of:

- Accelerometers to measure linear acceleration along the X, Y, and Z axes.
- Gyroscopes to measure angular velocity, detecting rotational movement.
- Magnetometers to find absolute heading relative to Earth's magnetic field.

By integrating accelerometer and gyroscope data, IMUs track changes in position and orientation over time. Advanced filtering algorithms, such as the Kalman filter, help fuse sensor data to improve accuracy and reduce errors. This measurement system is crucial for precise navigation, motion tracking, and control in various applications.

Without proper intrinsic calibration, even advanced systems may produce unwanted results, leading to inaccurate navigation and control. [10].

Common issues in IMUs

Additive-White-Noise [10]

Rapid variations in the sensor signal can be represented by a zero-mean, independent, continuous-time white Gaussian noise process $n(t)$, characterised by noise strengths. The larger these values, the “noisier” the gyroscope and accelerometer outputs.

Random-Walk [10]

Slow changes in sensor bias over time are modelled using a Brownian motion process, often referred to as a Wiener process. This behaviour is generated by integrating white noise with strengths (gyroscope) and (accelerometer).

Bias [10]

refers to a consistent offset in sensor readings that requires correction. Static Bias (Offset): A constant error that remains unchanged over time. Bias Instability: A dynamic error that evolves based on stochastic processes such as a first-order Gauss-Markov process.

Scale-Factor-Errors [10]

Scale factor errors arise when the proportionality between the output signal and the actual measured value deviates from the ideal. These errors are typically corrected by comparing the sensor readings to a known reference.

Cross-Axis-Sensitivity [10]

Misalignment between sensor axes introduces cross-coupling errors. These errors are quantified as misalignment angles, which can be corrected using a rotation matrix once determined.

Temperature-Effects

[10]

Sensor performance may degrade with temperature variations, causing increased bias or noise. High-precision IMUs often include internal temperature sensors and apply compensation algorithms to mitigate these errors.

2.3.1 Min-Max Normalisation [11]

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Sensor values can be normalised based on their max and min values, which will vary due to hand sizes.

Known Range: Flex sensors typically have a predictable resistance range, such as from 10 kΩ (flat) to 45 kΩ (fully bent), making Min-Max normalisation straightforward and effective.

Consistency: It maps the sensor output to a fixed range (e.g., [0, 1] or [-1, 1]), which is helpful when you need to interpret the sensor's data in a standard form for further processing. It's easy to implement and doesn't require complex calculations, making it ideal for real-time applications.

2.3.2 Zeroing and Drift Correction [12]

Any time the device is known to be completely stationary, the gyroscope offset can be nulled for that respective axis. This opportunity can vary wildly depending on the application. But any instance that the system is at a recurring resting state can be used for nulling, such as an idling car, a stationary autonomous robot, or the time between human footsteps. This can be used to remove drift in the gyroscope; however, it requires the algorithm to identify a stationary point.

2.4 Comparison of Similar RC gesture controllers

Product	Number of gestures	Gesture recognition (%)	Latency (ms)
Smart-Data-Glove-Based Gesture Recognition (SVM) [13]	35	92	36.7
A Machine Learning Approach to EMG-Based Hand Gesture Recognition for Interactive RC Car [14]	4	81.8	unknown
Machine Learning-Based Gesture Recognition Glove [15]	5	90	63
Glove-Based Hand Gesture Recognition for Diver Communication [16]	13	98	330

Table 1 – Existing models' performance

All the above gesture control systems were glove-based controllers implemented in different scenarios. All products shown in the table are from research-based papers. The models have ranging accuracies from 81.8-98 and latencies from 15ms to 330ms.

2.5 Proposed Solution

The project will aim to achieve a gesture recognition accuracy of around 98% whilst maintaining a low latency system of around 30ms. Multiple advanced gestures are to be implemented into this device and evaluated.

The project will use a glove-based gesture controller with Gyroscopes and flex sensors due to their low cost, high accuracy and lower power requirements as opposed to vision-based and capacitive-based sensors; To accurately capture gestures as data.

Calibration methods such as drift correction and min-max normalising are to be used to ensure readings from the sensors are highly accurate.

Finally, a suitable machine learning model will be selected (SVM, ViTs, and CNNs), with a focus on classification tasks, as this is central to the gesture recognition system. While Support Vector Machines (SVM) are widely used for classification, Vision Transformers (ViTs) and Convolutional Neural Networks (CNNs) will also be evaluated based on their ability to efficiently classify gestures with varying complexity. The selection of the model will also take into account the computational power available on the ESP32, ensuring that the chosen model can run efficiently on the microcontroller without compromising speed or accuracy. The selected model will be implemented into the gesture controller, trained to recognise gestures at high speed, thus minimising system latency and enhancing overall gesture recognition accuracy."

Chapter 3: Design And Method

This chapter outlines the approach used to develop the ESP32-based gesture controller for an RC car using neural networks. The chapter begins by describing the overall system architecture, highlighting the key components such as the ESP32 microcontroller, the gesture recognition system, and the communication between the controller and the RC car. It then delves into the hardware design, explaining the integration of gesture sensors, motor drivers, and control modules. The software design section focuses on the neural network model for gesture recognition, detailing its training process and deployment on the ESP32.

3.1 System Architecture

The flowchart (Figure 4) shows a simplified model of the system as a whole. The flowchart consists of two main blocks: the gesture controller and the RC car. The two blocks are connected via the ESP-NOW wireless communication protocol.

ESP-NOW works as a P2P protocol allowing for direct communication between two ESP-32 microcontrollers. The communication mode is set as one-way communication, as shown in the flowchart. The RC Car is set as the receiver and is identifiable by its unique MAC Address. The Gesture controller is set as the sender, it broadcasts to the receiver MAC address an encoded integer value which consists of a unique identifier for the gesture and speed.

The encoding scheme of the encoded integer sent follows a three-character format (XXY), where XX (Gesture Code) is the recognised gesture, assigned a unique two-digit identifier. Y (Speed Level): Is the speed of movement, ranging from 1 (slow) to 9 (fast).

Gesture/Action	XX (Gesture Code)	Description
Emergency Stop	0	Immediate stop (override all)
Tilt-Up	1	Drives forward
Tilt Down	2	Drives backward
Clockwise rotation	3	Turns Right
Anti-CW rotation	4	Turns Left
Stationary	5	Car remains still
Nod Gesture	6	To be determined
Wave Gesture	7	Flash Front lights
Shaking Gesture	8	To be determined
More to be added	9 - XX	...

Table 2 – Gesture mapping

Speed Level (Y)	Description
1 and 2	Very Slow
3 and 4	Slow
5 and 6	Medium Speed
7 and 8	Fast
9	Maximum Speed

Table 3 – Speed mapping

The speed control is used in gestures 'Move forward', 'Move backwards', 'Turn right' and 'Turn left'. The speed control is determined using a stationary (Open hand with palm faced down) as the reference point. The magnitude of the angle determines the speed, where the maximum speed is $\pm 69^\circ$ and higher (+)/lower (-). The exact speed level corresponding to the table above is found by an integer division of the obtained angle, subtracting 15 and divided by 6. This is to allow for 9 unique speed levels; values beyond $\pm 69^\circ$ are assigned 9 (maximum speed). Speed control is further developed in Chapter 3.

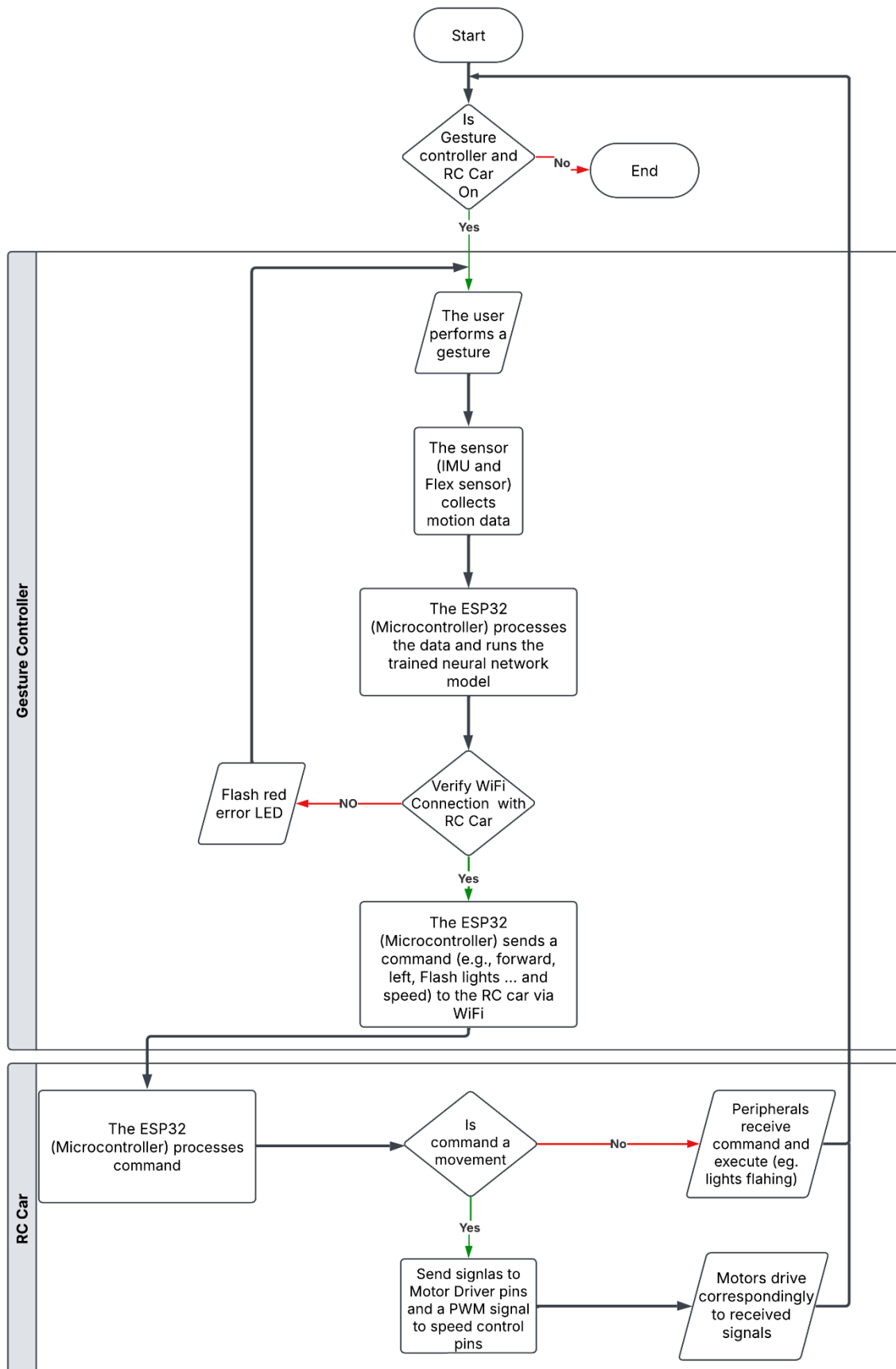


Figure 4. – System Workflow Diagram

3.2 Hardware Design

This section outlines the hardware design of the ESP32-based gesture-controlled RC car, covering circuit design, wiring diagram, and component selection. It details how various components, including sensors, microcontrollers and communication modules, are integrated to ensure reliable operation. The wiring diagram illustrates the physical connections, while the component selection explains the choice of hardware based on performance, compatibility, and efficiency.

3.2.1 RC Car design

Component	Quantity	Price (per unit)	Description
ESP32S	1	£6.99	ESP32-Wroom Microcontroller with Bluetooth and Wi-Fi communication.
L298N	2	£3.00	L298N Motor Drive Controller Board DC Dual H-Bridge
Motor	4	£1.40	Yellow gear motor dual shaft' 1.5V-9V
Li-ion Batteries	2	£1.17	18650 6800mAh 3.7V 20A Max Discharge Lithium-Ion batteries
18650 - 2 Battery Holder	1	£3.35	Battery holder for 18650 allows for two batteries to be connected, Switch is included
Wheels	4	£1.00	3cm by 7m Diameter Wheels Yellow and Black
Platforms	2	£1.16	2 Platforms Laser Cut at QMUL Electronics Lab
Total Price		£29.44	

Table 4 – RC Car component table

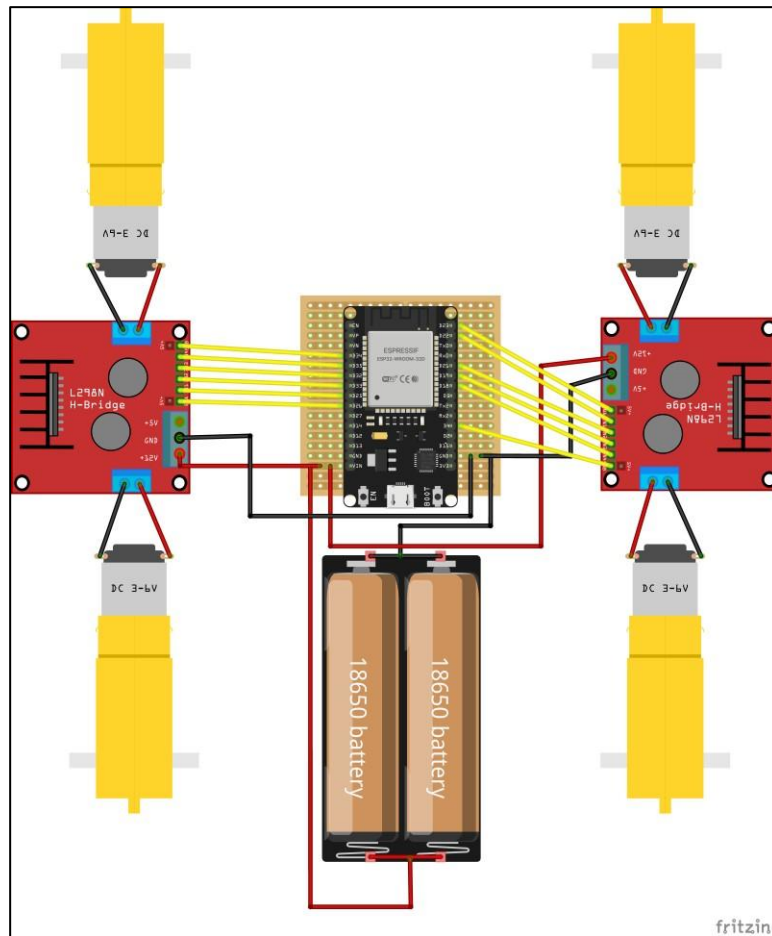


Figure 5. – RC car Circuit design (Made on Frizing)

Figure 5 shows the circuit diagram of the RC car. The circuit consists of 4 unique components (ESP32 Microcontroller, L298N Motor Drivers, 18650 3.7 V Lithium-Ion Batteries and Electric gear motors). A split was made down the middle of the stripboard to separate all the connection pins of the ESP32.

Two 18650 Li-Ion batteries (3.7 V, 6800 mAh capacity and 20 A max discharge; 1 battery) are used to power the car. The power rail can be located on the bottom left of the stripboard, and the ground rail is located on the bottom right of the stripboard. The 2 L298N Motor drives are connected directly to the power and ground rails and are supplied with the full 7.4 V. The microcontroller has a ground pin and a Vin pin, which are also connected to the power and ground rails. The ESP32 has a built-in voltage regulator; therefore, the input 7.4 V will be reduced to 3.3 V.

Connections: The yellow wires found on figure 5 are connections made from the output pins of the Microcontroller (Analogue/ digital pins) to the controlling pins of the L298N motor controller. The L298N has 6 controlling pins: ENA, ENB, IN1, IN2, IN3 and IN4.

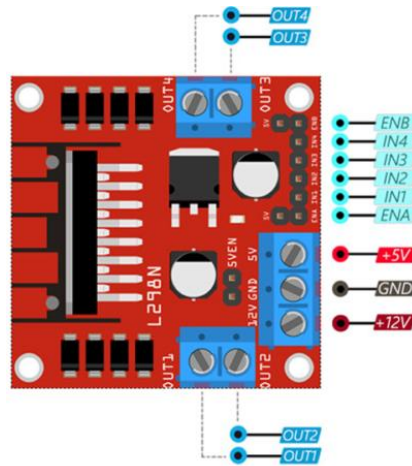


Figure 6. L298N Pin Layout.

Truth tables below show how the L298N is controlled (1 = High [3.3V] 0 = Low [0V])

L298N Motor Controller Truth Table (IN1 AND IN2)			
ENA	IN1	IN2	Description
0	N/A	N/A	Motor A is off
1	0	0	Motor A is stopped (Brakes)
1	0	1	Motor A is on and turning backwards
1	1	0	Motor A is on and turning forward
1	1	1	Motor A is Stopped

Table 4 – Motor controller truth table (ENA, IN1 AND IN2)

L298N Motor Controller Truth Table (IN3 AND IN4)			
ENB	IN3	IN4	Description
0	N/A	N/A	Motor B is off
1	0	0	Motor B is stopped (Brakes)
1	0	1	Motor B is on and turning backwards
1	1	0	Motor B is on and turning forward
1	1	1	Motor B is Stopped

Table 5 – Motor controller truth table (ENB, IN3 AND IN4)

For example, if ENA = 1, IN1 = 1, IN2 = 0, ENB = 1, IN3 = 1 and IN4 = 0, both motors A and B connected to the Motor driver will move forward. There are 4 motors connected to the motor controllers, which are supplied with voltage through the motor control at a peak of 7.4 V.

ENA and ENB are also used for speed control. This is done by varying the voltage sent to the controlling pins (0 – 3.3 V). The ESP32 can produce PWM signals, which allows the user to vary the voltage to the ENA and ENB pins. The output voltage can be found using the equation below.

$$V_{Out} = V_{in} \times D$$

D is the corresponding duty cycle of the PWM signal, and V_{in} is the Voltage supplied directly to the L298N through the 12 V+ OR 5 V+ Pins. For example, if we had an input voltage of 7 V and a duty cycle of 50%, the output voltage would be 3.5 V. The PWM signal is fed directly into the H-bridge switches/ MOSFETS.

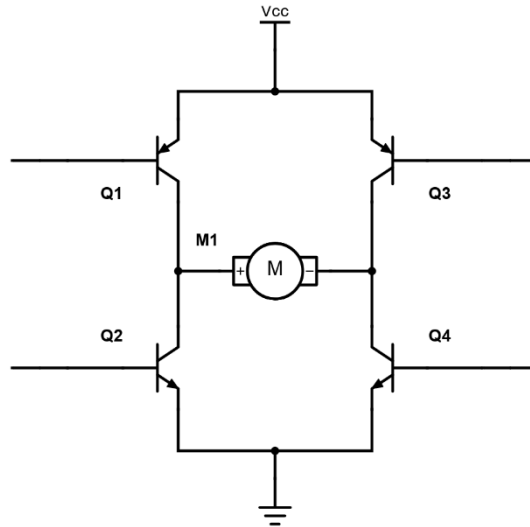


Figure 7. Single H-bridge

The L298N is made up of two H-bridges, the switches(MOSFETS) are controlled by the previously explained PWM signal and the direction is controlled by IN1 and IN2 (For the 1st H-bridge) and IN3 AND IN4 (For the 2nd H-bridge), shown in the Truth tables (Table 3 and 4). The MOSFETS work in pairs, Q1 and Q3, Q2 and Q4; if both pairs are to be closed simultaneously, the Vcc is shorted directly to ground. When Q1 and Q4 are closed, current flows from the left to the right of the motor (through Q1 and Motor +, out of Motor –, then to ground through Q4). When Q3 and Q2 are closed, current flows in the opposite direction (Through Q3 and Motor-, out of Motor+, then to ground through Q2). This further explains the truth tables 3 and 4, where IN1 and IN2 control which pairs of switches are to be closed, IN1 correspond to Q1 and Q3, and IN2 corresponds to Q2 and Q4.

IN3, IN4 and ENB work exactly like IN1, IN2 and ENA as described above, but they control a separate H bridge; effectively, the system consists of 2 H-bridges, each used to control 2 separate motors simultaneously.

3.2.2 Gesture Controller design.

Component	Quantity	Price (per unit)	Description
ESP32S	1	£6.99	ESP32-Wroom Microcontroller with Bluetooth and Wi-Fi communication.
Flex sensors	4	£6.49	Variable resistive strips that increase in resistance when bent.
Adafruit BNO085 IMU	1	£23.65	9-axis IMU (Accelerometer, Gyroscope and Magnetometer), with a built-in fusion program allowing for Rotation angles to be obtained directly from the board without calculations.
Gloves	1	£3	To place the components.
Total Price		£59.60	

Table 6 – Gesture Controller component table

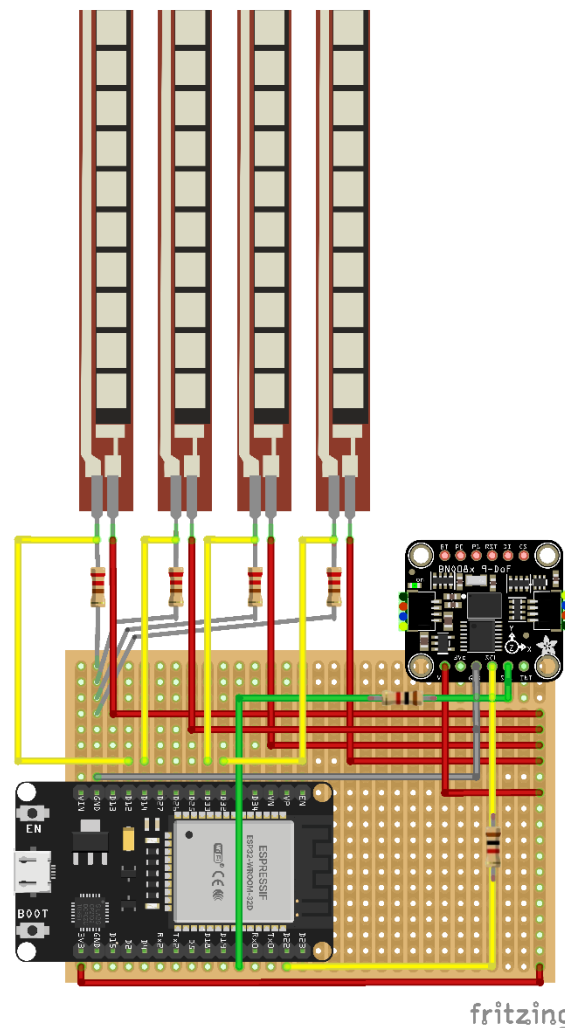


Figure 8. Gesture Controller Circuit Diagram

Figure 8 shows the complete circuit diagram of the gesture controller. Red wires are connections to the supply voltage (3.3 V), grey wires connect to ground (GND), while yellow and green wires are signal lines connecting to ESP32 input ports.

The ESP32 has two dedicated pins for I²C communication: the Serial Data Line (SDA) and Serial Clock Line (SCL), which are connected directly to the Inertial Measurement Unit (IMU), allowing for synchronised bidirectional data transmission.

The controller is powered via the USB-C port, supplying 5 V to the ESP32 development board. An onboard 3.3 V regulator (AMS1117-3.3) provides the stable voltage required for both the ESP32 and the BNO085 IMU.

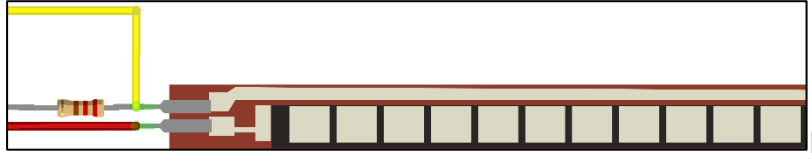


Figure 9. Flex Sensor Module

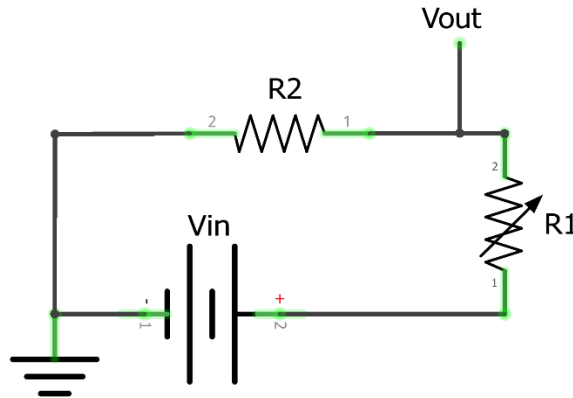


Figure 10. Flex Sensor module equivalent circuit.

Each flex sensor forms a voltage divider with a fixed 10 kΩ resistor, where the flex sensor operates as a variable resistor. The resistance of the flex sensor is approximately 45kΩ when unbent and decreases to around 1 kΩ as it bends. The output voltage from each voltage divider is given by:

$$V_{out} = V_{in} \frac{R2}{R1 \times R2}$$

Where R1 represents the flex sensor resistance, R2 is the fixed 10 kΩ resistor, and Vin is the supply voltage of 3.3 V. This configuration allows the analogue voltage output to vary between approximately 0.6 V when the sensor is straight and 3V when fully bent, remaining comfortably within the ESP32 ADC input range of 0–3.3 V.

The choice of a 10 kΩ fixed resistor was made to maximise sensitivity in the mid-range of the flex sensor's operation while maintaining a low output impedance. A low impedance is critical to ensure accurate and stable analog-to-digital conversions, minimising the effects of the ESP32 Analogue-to-Digital Converters (ADC) inherent non-linearity. The ESP32 microcontroller features a 12-bit ADC, which provides a resolution of approximately 0.8 mV per ADC count. This high resolution enables fine-grained detection of finger bending angles, which is essential for precise gesture recognition.

The Inertial measurement unit (**Adafruit BNO085 IMU**) comes with built-in sensor fusion, all sensor data is combined to give highly accurate yaw, pitch and roll angles. The sensor sends the Yaw, Pitch and roll angles constantly to the ESP32 microcontroller through the Serial Data Pin. The sensor also has a built-in 100 nF decoupling capacitor, which suppresses high-frequency switching noise from affecting sensitive sensor measurements.

The I²C bus requires pull-up resistors on the serial data line and serial clock line, as it uses an 'open collector' design for communication. This means that the device can pull the line low by connecting to ground, and the resistors are used to pull the line to logic high (3.3 V).

The I²C bus is set to 'FAST MODE', which is 100-400 KHz. This is matched to the top frequency of the BNO085 (250 KHz). The required rise time in fast mode is less than 300 ns [17]. Factors affecting the rise time include pull-up resistor values and bus capacitance. The bus capacitance is 22 pF (Found in Appendix I). Bus capacitance refers to the capacitance in lines SCL and SDA.

$$R_p = \frac{t_r}{0.8473 \times C_b} \quad [17]$$

To calculate the required pull resistor value, the equation above is used, t_r = Rise time, R_p = Pull-up resistance and C_b = bus capacitance. (22 pF and 300 ns are substituted into the equation)

$$R_p = \frac{300 \times 10^{-9}}{0.8473 \times 22 \times 10^{-12}} = 16093.9\Omega$$

Therefore, the maximum resistance of the pull-up resistors can be 16093.9Ω. The SDL and SCL pins also have a minimum current sink of 3 mA [17]. So, the maximum possible resistance that can be used to meet both current and rise time requirements is $\frac{3.3V}{3\text{ mA}} = 1.1\text{ k}\Omega$. The resistance values chosen were 1 kΩ Due to availability, resulting in a rise time of 18 ns and an available current of 3.3 mA.

The data received will have drift; therefore, mean bias calibration is required to be implemented. (Explained further in the report)

3.3 Gesture Recognition Model Design

This section covers the architecture of the neural network, data collection, data preprocessing, and training process. It details how input gesture data is structured, processed, and used to train the model, ensuring accurate classification for controlling the RC car.

3.3.1 Data Collection

Collecting data is a key component of creating a machine learning model. Collecting data directly from the gesture controller is explained by the Pseudocode below.

```
BEGIN

  Initialize ESP32 and sensors

  VOID loop()
    WAIT for button press

    IF button is pressed THEN
      Start data collection timer (1000ms)

      WHILE timer < 1000ms DO
        Read current timestamp
        Read sensor values (Yaw, Pitch, Roll)
        Serial print: timestamp, Yaw, Pitch, Roll

        WAIT for 10ms // Wait to collect data at 100Hz
      END WHILE

      RESET timer to 0 // Ready for next button press
    END IF

  END VOID loop()

END
```

Figure 11. Data collection Pseudocode

A pull-up resistor button is connected to the gesture controller. It is connected to Ground, Voltage and input pin of the ESP32 (From 3.2.2 Gesture Controller design). When the button is pressed, the signal at the input pin is high; otherwise low. The button press is used to start the data collection.

A timer is used to determine the current timestamp, this is to ensure the data is of the same duration. The length of the timestamp is 1000ms or 1s, and the data is collected at a frequency of 100 Hz, meaning every sample has a 10-millisecond gap with a total of 100 samples. Each sample consists of the timestamp (ms), yaw (°), pitch (°) and roll (°). Data is sent to the Serial Monitor through a serial communication protocol between the microcontroller and the computer. The microcontroller uses the Serial. Print() or Serial.println() commands to transmit data, such as sensor readings, over a USB or serial connection. The data is saved as a Text document. This approach allows for quick and easy data collection.

Timestamp (ms)	Yaw (°)	Pitch (°)	Roll (°)
10	-15.81	31.88	-0.38
20	-15.55	31.83	-0.21
30	-15.23	31.77	0
40	-14.81	31.68	0.25
50	-14.28	31.53	0.53
60	-13.64	31.3	0.83
70	-12.91	30.93	1.14
80	-12.17	30.53	1.44
90	-11.36	30.04	1.76
100	-10.46	29.44	2.11
110	-9.53	28.77	2.47
...	...	...	...
900	20.45	-1.42	10.57
910	19.16	0.16	10.67
920	17.77	1.86	10.67
930	17.07	2.7	10.63
940	15.67	4.39	10.48
950	13.51	6.91	10.2
960	12.12	8.49	10
970	10.81	9.86	9.89
980	9.45	11.23	9.83
990	8.07	12.62	9.79

Figure 12. Text Format for data representation

This is the resultant format of the data collection. This shows time series data of yaw, pitch and roll, representing a singular sample. Hundreds of samples are taken [with slight variations in the gestures, which](#) consist of the speed of the gesture and varying the direction the gesture controller is facing. This is required as the speed of the gestures in real applications will always have slight variations. Performing the gestures in different directions is related to the usage of the controller in an RC Car; the user may follow the car and may turn their body around, this shift causes variations in the sensor readings. Hundreds of samples are taken for each gesture to ensure a large enough dataset is available to train the model.

3.3.2 Data Preprocessing (Using Edge Impulse Software)

The raw data must be represented in a way such that the data can be easily identified as clusters, each cluster will represent a separate gesture. This requires analysis of time series and spectral power graphs and identifying features that can easily separate data into identifiable groups (wave, Tsunami and comsi). The terms Wave, Tsunami and Cosmi are code names representing specific gestures used for controlling the RC car.

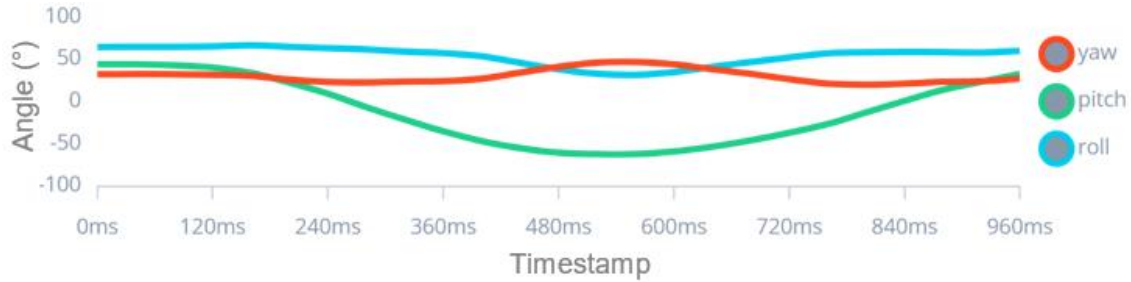


Figure 13. Time series of raw data of the Wave sample

A Fourier Transform is applied to the raw data time series, converting the time-domain data into the frequency domain. This process results in a spectral power graph, which illustrates the distribution of signal power across different frequencies. No filtering was done on the raw data as the waveforms were relatively smooth. The power/ energy axes are logarithmically transformed.

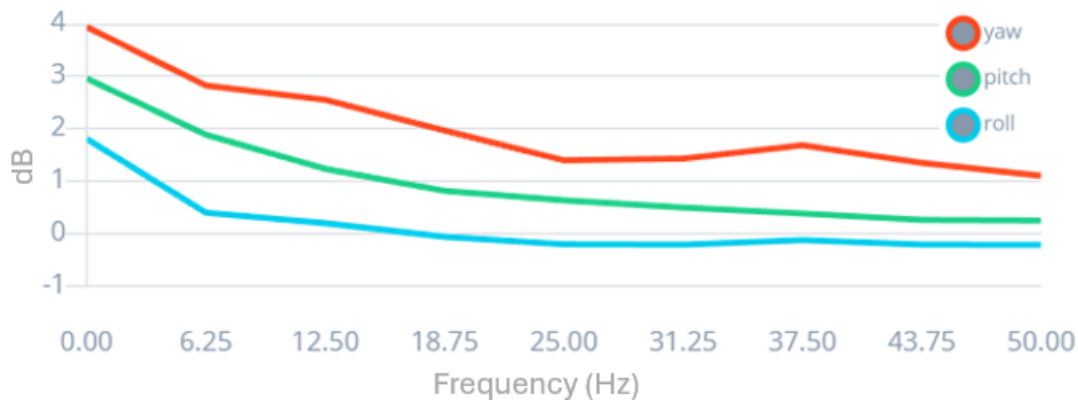


Figure 14. Fourier Transform - Spectral power (Log) graph of Wave sample

Both time series and spectral power graphs play a crucial role in identifying key features of data. Using the time series, we can obtain the RMS Yaw, pitch and roll values. Through the spectral power graph, we can obtain the power in different frequency bands, allowing us to identify the importance of each frequency band.

The edge impulse software allows for direct analysis of all samples, allowing us to find the similarities in features between all the data samples. Edge impulse calculates the feature importance (The features are RMS values and Power at specific frequency bands). The feature importance list will help identify the axis of the data representation graph, also known as the feature plots. The axis will be 3-dimensional, consisting of 1 feature from each Yaw, Pitch and Roll. The process of generating features will reduce the amount of signal analysis required on the device. The Edge impulse software uses a RandomForestClassifier to identify the feature importances.

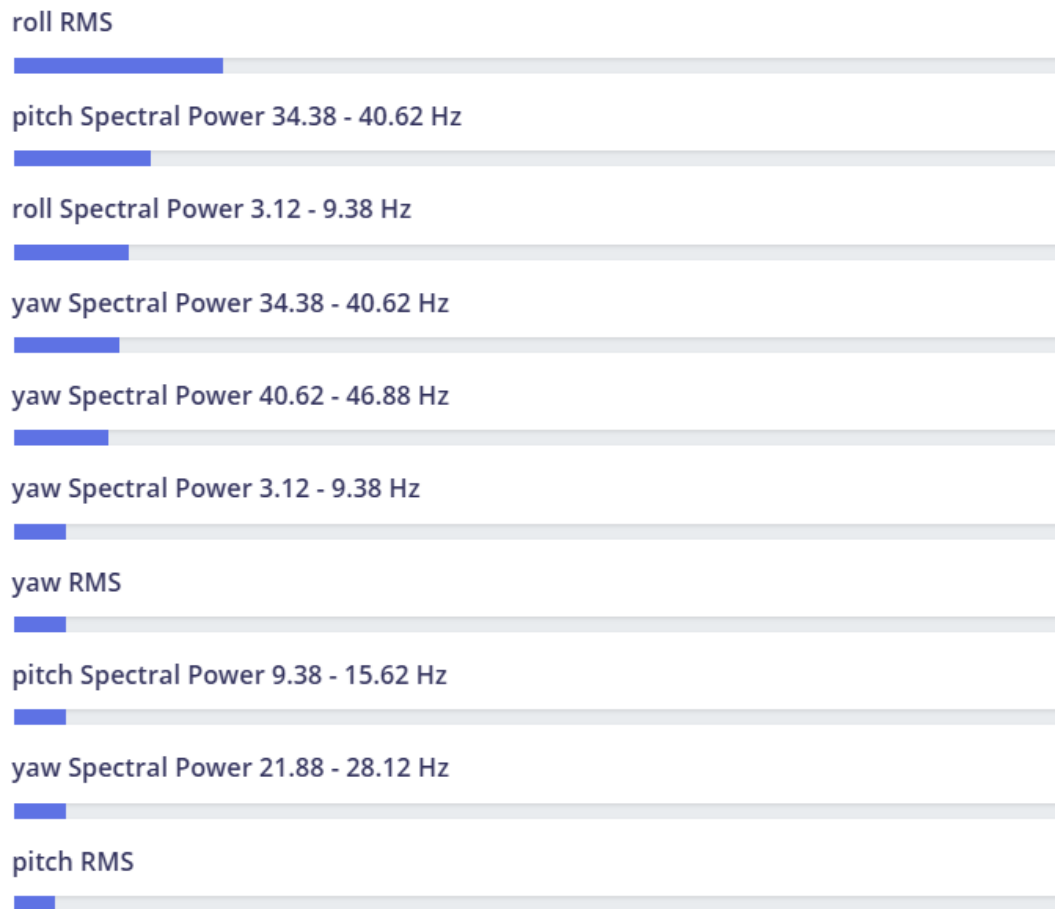


Figure 15. Feature importance

The output of the RandomForestClassifier gives each feature and its importance throughout all the datasets. The feature importance has no numerical values assigned to it, rather, it gives the user visual feedback as to which features are of importance (The longer the blue line, the more important). Using this, Feature plots axes are made up of the features listed above, but always including 1 feature from Yaw, Pitch and Roll.

To ensure that the data is ready to be used, we must find a Feature plot where each gesture's data points are clustered together with no/ very minimal overlapping of clusters. In the current scenario, as 3 gestures are being trained, there should be 3 distinguishable clusters of data. This also allows for identification of anomalous results, assuming that the data is well clustered, any data points that are far beyond their corresponding cluster will be noted as anomalous.

Figure 16 shows the result of all the Feature analysis and the axes used, where the RMS values of the time series of Yaw, Pitch and Roll. Each data point is characterised by the three mentioned features. This Feature plot has 3 distinguishable regions for Wave, Tsunami and Comsi with no overlapping between the clusters. However, the data points for the comsi and tsunami are a lot more spread out. As a result of this, the model may need to learn more complex boundaries to classify the data correctly. For example, the boundaries will have non-linear shapes and will require more advanced techniques to model effectively.

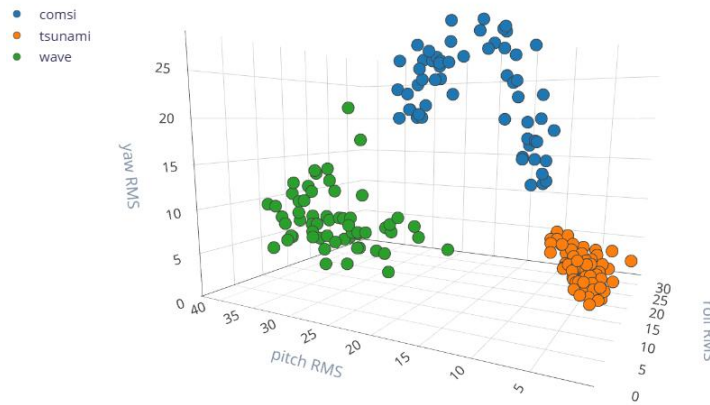


Figure 16. Feature plot (RMS Yaw, Pitch and Roll)

Figure 17 shows a separate set of axes; the Spectral power of pitch between 34.38 Hz and 40.62 Hz is used rather than RMS pitch. This feature is obtained from the Spectral power graphs, and using the feature importance finding it was the feature with the 2nd most importance. This representation also has distinguishable features; however, the data points are closer together compared to Figure 16. Well-clustered data points and no overlapping of data points are key for an efficient and simple neural network, it will work with less complex regions and more linear-shaped regions.

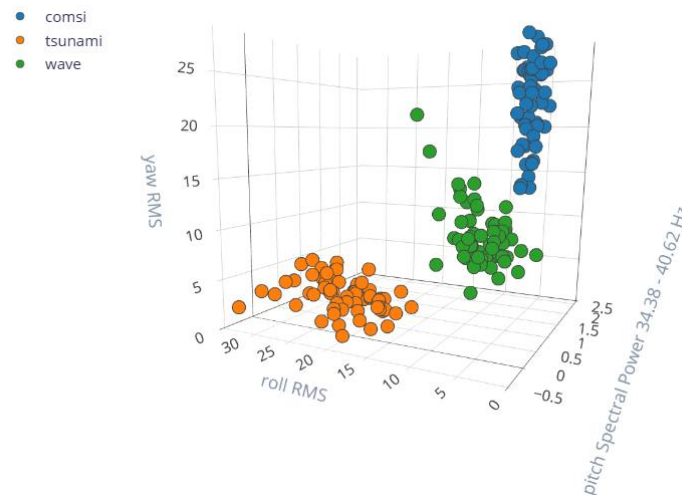


Figure 17. Feature Plot (RMS Yaw, RMS roll and pitch Spectral power 34-40 Hz)

Figure 17 was utilised in the training of the neural network classifier as it represents the data more clearly than Figure 16. And the lack of complexity in the cluster regions will reduce the complexity of code and required computational power, and most importantly, the processing time, which has a direct effect on the latency of the device.

3.3.3 Training the neural network model

This section describes the Creation and testing of the neural network through the flowchart and a breakdown of the entire code. The model is built using the Sequential API, starting with an input layer, followed by a dropout layer, a hidden layer, and a softmax output layer for multi-class classification. It uses the Adam optimiser with specified parameters and categorical cross-entropy for loss. A custom callback logs training progress. After training, a quantisation step reduces model size and improves efficiency. This approach balances learning, optimisation, and generalisation..

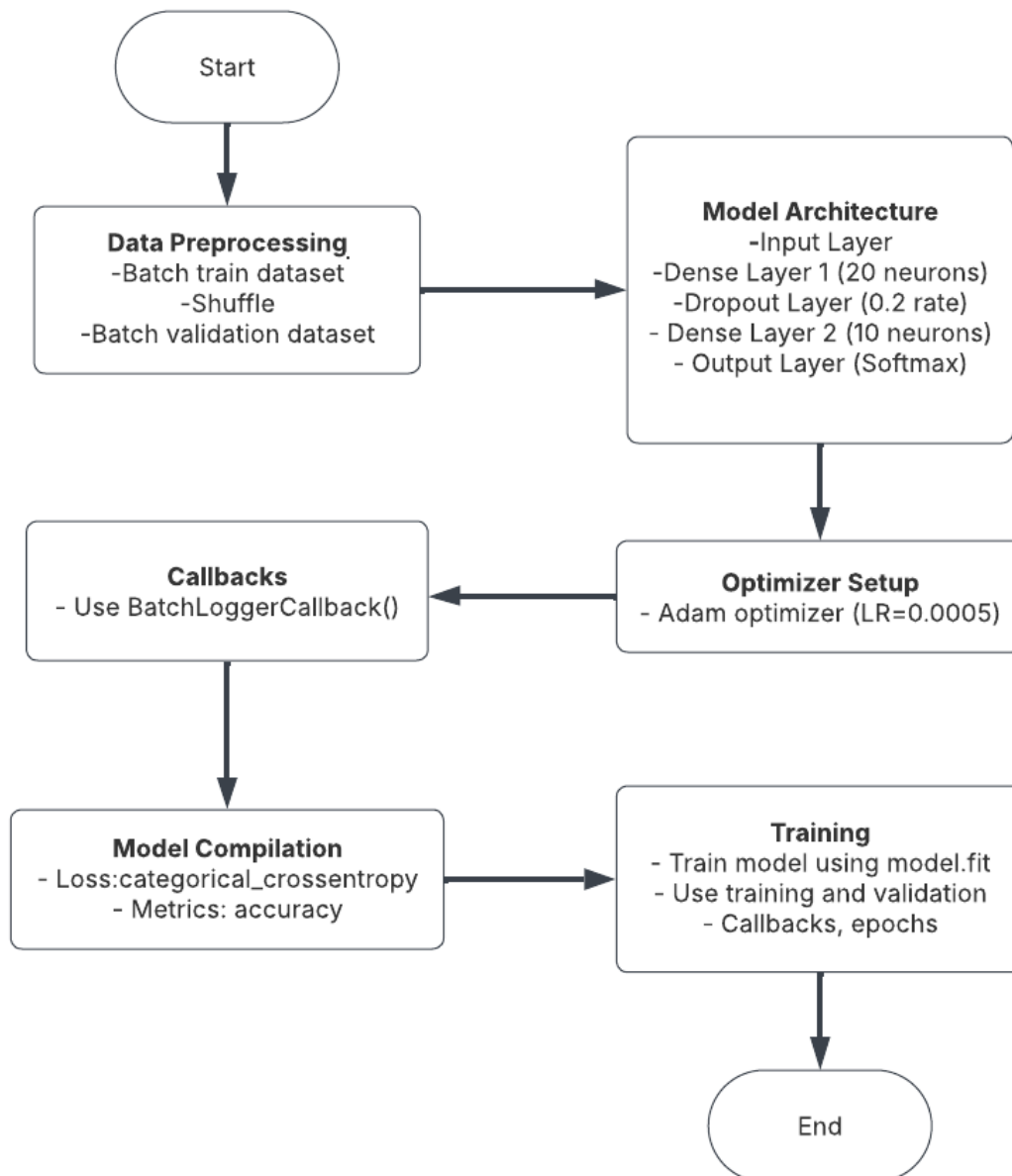


Figure 18. Creation and training Flowchart of a neural network

The code starts by importing the required libraries. TensorFlow is the deep learning framework used. Sequential API is used to build a simple feedforward neural network. Dense layers define fully connected layers. Dropout is used to prevent overfitting. Adam Optimiser (legacy version) is chosen for training.

```
import TensorFlow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

from tensorflow.keras.optimizers.legacy import Adam
```

Figure 19. Neural Network Library Importing

After defining the model architecture, several hyperparameters were selected based on the complexity of the gesture recognition task and the relatively small input feature size (39 features). The model was trained for 30 epochs, balancing sufficient learning with avoidance of overfitting, as training beyond this point showed minimal performance improvement during testing. A learning rate of 0.0005 was chosen to ensure slow, stable convergence, preventing the optimiser from overshooting the minimum in the loss function. The batch size was set to 32, a common choice that offers a good compromise between computational efficiency and stable gradient updates.

```
epochs_to_train = args.epochs or 30

learning_rate_value = args.learning_rate or 0.0005

ensure_determinism_flag = args.ensure_determinism

batch_size = args.batch_size or 32
```

Figure 20. Neural Networks Hyperparameters

If determinism is not enforced, the training data is shuffled to improve generalisation. Batching is performed on training and validation datasets to optimise GPU/CPU processing.

```
if not ensure_determinism_flag:

    train_dataset = train_dataset.shuffle(buffer_size=batch_size * 4)

    train_dataset = train_dataset.batch(batch_size,
                                        drop_remainder=False)

    validation_dataset = validation_dataset.batch(batch_size,
                                                  drop_remainder=False)
```

Figure 21. Neural network data shuffler

The network architecture was deliberately kept small to match the relatively low dimensionality and variability of the gesture input data. The first hidden layer uses 20 neurons with ReLU activation, providing sufficient capacity to model nonlinear relationships without unnecessarily increasing model complexity. L1 regularisation (coefficient 0.00001) was applied to encourage sparsity in the weights and reduce overfitting. A dropout layer with a dropout rate of 0.2 further improves generalisation by preventing co-adaptation of neurons. The second hidden layer uses 10 neurons, again with ReLU activation and L1 regularisation, to progressively reduce the feature space before final classification. The output layer uses Softmax activation appropriate for multi-class classification, with the number of neurons equal to the number of gesture classes. This compact architecture was sufficient for the relatively simple classification task.

```
model = Sequential([
    Dense(20, activation='relu',
        activity_regularizer=tf.keras.regularizers.l1(0.00001),
        Dropout(0.2), # Added Dropout layer with rate 0.2
    Dense(10, activation='relu',
        activity_regularizer=tf.keras.regularizers.l1(0.00001),
    Dense(classes, name='y_pred', activation='softmax')
])
```

Figure 22. Neural network layer architecture

The Adam optimiser was used to adjust the network weights during training. A learning rate of 0.0005 was selected to ensure gradual and stable convergence. The optimiser's hyperparameters β_1 and β_2 were set to 0.9 and 0.999, respectively, controlling the momentum and the rate of adaptation of the learning process. These standard values were chosen as they provide a good balance between fast convergence and avoidance of instability.

```
optimizer = Adam(learning_rate=learning_rate_value, beta_1=0.9,
beta_2=0.999)
```

Figure 23. Neural network optimiser

Custom Callback (BatchLoggerCallback) logs training metrics per batch. Helps in monitoring performance.

```
callbacks.append(BatchLoggerCallback(batch_size, train_sample_count,
epochs=epochs_to_train, ensure_determinism=ensure_determinism_flag))
```

Figure 24. Neural network callback function

Loss Function: categorical_crossentropy, Suitable for multi-class classification.
Evaluation Metric: Accuracy is used to measure model performance.

```
model.compile(loss='categorical_crossentropy', optimizer=optimizer,  
metrics=['accuracy'])
```

Figure 25. Model performance function.

model.fit() starts training the model: Uses the training dataset. Runs for epochs_to_train (default 30 epochs). Uses a validation dataset for evaluation. Callbacks log batch performance, display detailed training progress.

```
model.fit(train_dataset, epochs=epochs_to_train,  
validation_data=validation_dataset, verbose=2, callbacks=callbacks)
```

Figure 26. Neural Network training

The model is successfully trained, the resultant accuracy is 100% with a loss of 0.09, an accuracy of 100% suggests the model may be overfitting (learning too well from the data). Therefore, the dropout rate from the dropout layer was increased; the dropout layer will disable a percentage of the neurons to prevent the Neural network from overfitting. The models' training performance with a dropout rate of 0.5 was also 100% accurate, but with a loss of 0.33.

The probability confidence can be calculated for each approach using the cross-entropy equation (base. e), $Loss = \sum_{k=1}^K y_k \times -\log(P_k)$. y_k is the true probability of class 'k', which is represented by 1 for the correct class and 0 for all other classes [1 0 0]. 'K' represents the total number of classes, in this case 3. ' P_k ' represents the predicted probability of class k [P_1 P_2 P_3]. The equation simplifies to $Loss = -\log(P_1)$ Where P_1 is the correct class, therefore $P_1 = e^{-loss}$.

The probability of 'correct class' for a dropout layer of 0.2 is $e^{-0.09} = 0.914$ (91.4%), and for a dropout layer of 0.5 is $e^{-0.33} = 0.719$ (71.9%). Therefore, the model has a higher confidence at a 0.2 dropout rate. The significant drop-off in confidence at a 0.5 dropout rate suggests that the dataset is not of sufficient size for a higher dropout rate. A dropout rate of 0.2 was chosen, as it balances regularisation without overly diminishing model performance.

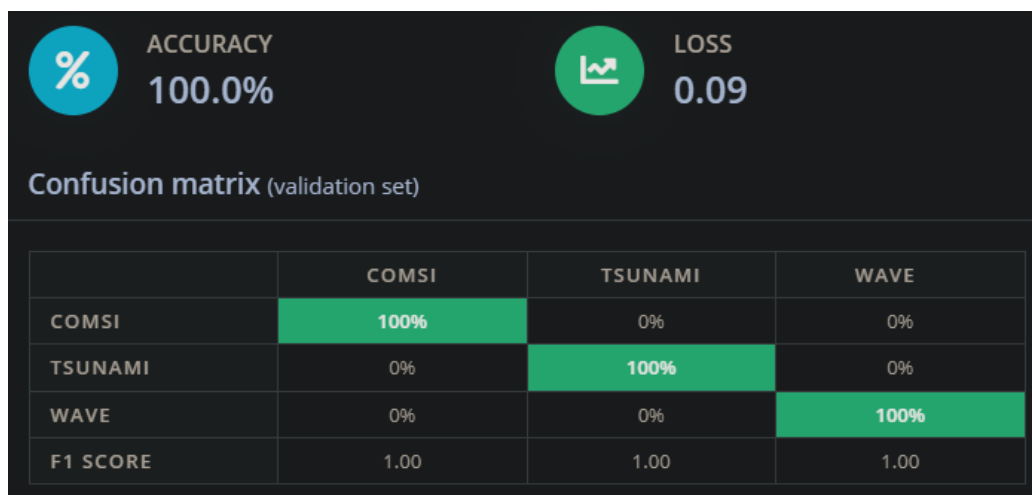


Figure 27. Accuracy, Losses and Confusion Matrix Table (Dropout rate of 0.2)

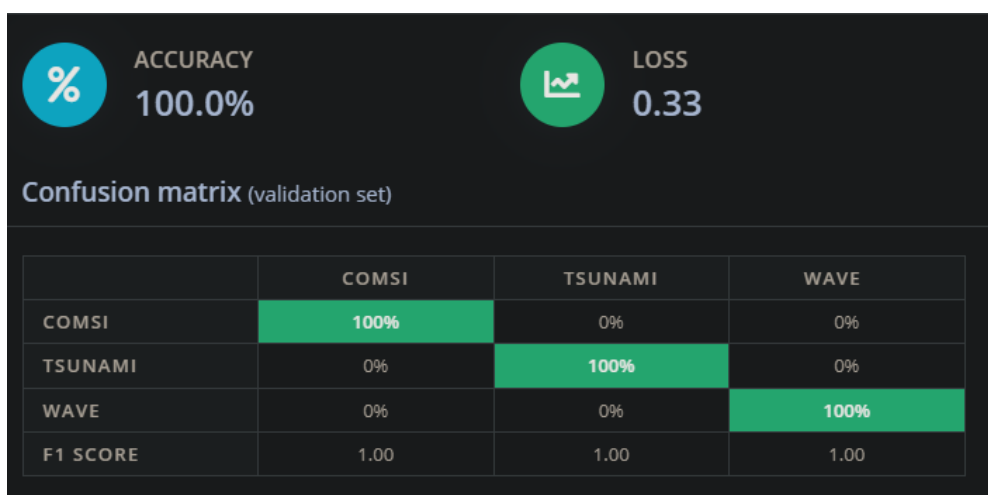


Figure 28. Accuracy, Losses and Confusion Matrix Table (Dropout rate of 0.5)

Chapter 4: Implementation

This chapter provides a detailed overview of the technologies used to implement the system and explains how its core functionalities were achieved. Additionally, it outlines the steps taken to ensure successful deployment.

4.1 Languages

The development of the ESP32-based gesture-controlled RC car relies on two primary programming languages: C/C++ and Python.

4.1.1 C/C++

C/C++ is used for programming the ESP32 microcontroller due to its efficiency, low-level hardware access, and real-time processing capabilities. It enables direct interaction with sensors, manages wireless communication, and controls the actuators that drive the RC car.

4.1.2 Python

Python is used for training the neural network that classifies gestures based on sensor data. With its powerful machine learning libraries like TensorFlow and Keras, Python allows for efficient data preprocessing, model development, and validation.

4.2 Essential Libraries

This section discusses the essential libraries and services used in the development of the ESP32-based gesture-controlled RC car.

4.2.1 Gestures_inferencing.h

This custom header file contains functions and algorithms for gesture classification and inference based on data collected from the sensors. It interfaces with the machine learning model to classify gestures and trigger the corresponding commands for the RC car's movement.

4.2.2 Arduino.h

The core library for all Arduino-based projects, including those involving the ESP32 microcontroller. Provides essential functions for hardware initialisation, input/output operations, and control flow. It enables the execution of basic operations like setting pins, serial communication, and handling timing events within the system.

4.2.3 Wire.h

A standard library for I2C communication, enabling the microcontroller to interface with I2C-compatible devices such as IMUs. The library simplifies the process of exchanging data between the ESP32 and connected I2C peripherals, facilitating efficient data transmission for real-time gesture tracking.

4.2.4 Adafruit_BNO08x.h

This library interfaces with the Adafruit BNO08x sensor, which is used for accelerometer, gyroscope, and magnetometer data collection to detect user gestures. The sensor provides the raw data (yaw, pitch, roll) required for gesture recognition, and this library manages communication with the sensor to retrieve and process that data.

4.2.5 esp_now.h

ESP-NOW is a low-power wireless communication protocol designed for short-range, peer-to-peer communication. This library allows the ESP32 to send commands and data to other ESP32 devices or the RC car over Wi-Fi without requiring an internet connection, ensuring fast and reliable communication between the components of the system.

4.2.6 WiFi.h

It enables the ESP32 to connect to a local network or communicate directly with other devices via Wi-Fi, making it possible to send commands to the RC car and manage remote control features. This is essential for ensuring that the system can wirelessly transmit gesture data and receive feedback from the car.

4.3 Development of Core Functionalities

Core Functionalities are explained, with included code snippets. The code is explained and how it is used in the system.

4.3.1 Closed-loop Mean Bias Calibration (Appendix B and C)

The approach used to correct the sensor's initial offset and built-up drift during usage was closed-loop Mean bias calibration. The initial offsets are calculated by having the user's hand open horizontally with their palm faced down (stationary). These offsets are applied to the Yaw, pitch and roll values. The system is considered stationary when the yaw, pitch, and roll values all remain within the range of -15° to $+15^\circ$.

During the usage of the gesture controller, whenever 50 (0.5ms, as each loop of the code is 10ms) consecutive stationaries are detected another Mean bias calibration (Fast mean bias calibration) is performed; But with a significantly reduced sample size of 5 as apposed to the 500 samples, this is to ensure that there are minimal effects on latency. The results of the 'Fast Mean bias calibration' are added to the existing offsets (offset correction), removing any built-up drift during usage.

Once a Fast mean Bias calibration is performed, there is a 2.5-second delay before another one can be performed. This is to prevent an infinite calibration loop.

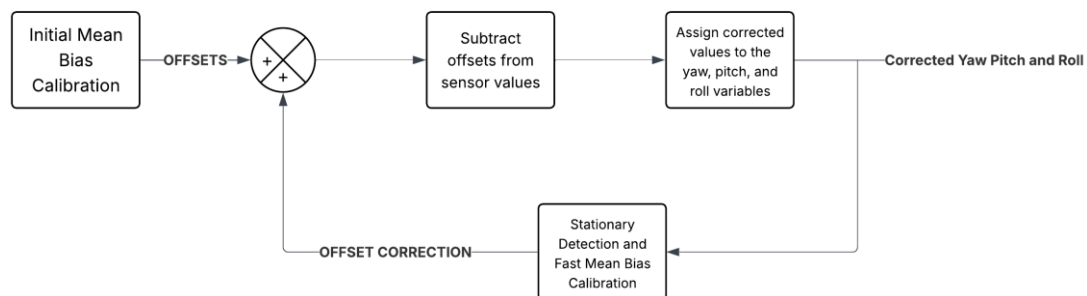


Figure 29. Closed-loop Mean Bias Calibration.

Appendix B: The Initial mean bias calibration code is presented. This consists of a loop that sums the total of 500 stationary readings with no applied offset and then finds the average value by dividing by 500. It then assigns the calculated offset values to the variables OffYaw, OffPitch and OffRoll. These offsets are then used in the main loop to correct the yaw, pitch and roll angles.

Appendix C, we see the Fast mean bias calibration works in the exact same way as the initial calibration, but with only 5 stationary readings and the applied offset. The resultant averages are assigned to a temporary variable only used within the function called FastPitch, FastRoll and FastYaw. The temporary variables hold the average offset correction value, which is added to the variables OffYaw, OffPitch and OffRoll.

Appendix E, the **else** statement containing 'stationary', validates whether 50 consecutive stationaries are detected and if the 2.5-second delay isn't enforced. If the conditions are met, the Fast offset function is called, and the delay counter is started.

4.3.2 Neural network implementation (Appendices D to F)

The code in **Appendix D** shows how the three example gestures are classified using the neural network. `results.classification[x].value` consists of integer values relating to the similarity to gesture. `results.classification[x].value` ranges from 0-0.99, where $X = 0$ represents the similarity with gesture 'comsi', and $X = 2$ represents the similarity with the 'wave gesture'. The sum of all classification values is 1. All the if statements are structured the same way. If the classification value for a corresponding gesture is greater than 0.70, the code enters an if statement for the detected gesture. It then increments the Gesture count value 'Wavec', 'Tsuc' or 'Comsic'. The code will skip an iteration of the main loop to prevent any interference. This is done because, for example, a wave requires the user to perform an upward tilt of the hand, which also moves the car forward, to prevent any unwanted movements. When the gesture counter reaches 3 and 2 specifically for 'comsi', the loop continues. The counters are reset every loop when the corresponding classification value is less than 0.7.

The code in **Appendix E** will only enter the IF statement if NONE of the previously explained gestures have a count of less than 3, 3, and 2 corresponding to the code. Once the code enters the if statement, it will check the four conditions where there is a clockwise or anticlockwise tilt (left or right), then an upward or downward tilt (Forward or reverse) and stationary if none of the conditions are met. It will assign a corresponding `send_value` for the detected gesture described in Table 2 – System Architecture. Each directional gesture also sets the stationary counter to 0 to maintain an accurate count on repeated stationaries. The stationary detection part of the code also shows the previously explained fast calibration. All of these gestures also have a speed attributed to them, which is calculated in the speed function.

The piece of code in **Appendix F** follows on from **Appendix E**, the code checks whether the gesture counts have reached their threshold for activation 3 for 'Tsunami', 3 for 'wave' and 2 for 'comsi', if the statement is true, the corresponding gesture that is activated will be assigned to the `send_value`. And only one will be active at once.

4.3.3 Remote Control Car Code (Appendix G)

The Car is set to a receiver, and the `send_value` is received from the gesture controller in the form XXY (explained in 3.1 – System architecture). The value is broken down into XX = gesture and Y = speed. The code consists of IF statements classifying gestures from 0 to 10. The IF statements contain motor speed, motor rotation direction and peripheral activation (LEDS).

4.3.4 Speed evaluation code (Appendix H)

The speed evaluation function is implemented in the Gesture controller code; the reasoning behind the selected values and limits is explained in 3.1 – System architecture. The code consists of a case statement that checks the corresponding `send_value`. The `send_values` are tied to the specific gesture performed. Speed control is only implemented into forward, left, right and reverse movements and adds a value 1-9 to the send value (XXY), where 1-9 refers to the speed (explained in 3.1 – System architecture)

Chapter 5: Testing And Results

5.1 Gesture Controller Performance Testing

This section provides test results. The documentation of the testing can be found in the supporting materials (Excel spreadsheet). Each gesture was tested 100 times, and the parameters tested are shown in Table 7.

Gesture	Accuracy (%)	Latency (ms)	Uncertainty (ms)	Angle (°)	Uncertainty (°)
Forward	100	31.5	±12		
Reverse	99	32.08	±9.5		
Left	100	30.98	±9.5		
Right	100	31.47	±10		
Stop	100	26.7	±5.5		
Wave	97	105.71	±14.5		
Tsunami	98	105.36	±14	90	±5
Comsi	89	105.5	±14.5	90.48	±4
Signal Right	100	32.87	±8		
Signal Left	100	30.85	±9		

Table 7 – Parameter testing results (Using range uncertainty)

The gesture '**Wave**' flashes front lights. The gesture '**tsunami**' is a rapidly tilting hand up and down, and results in a 90-degree turn right. The gesture '**comsi**' is a rapidly rotating hand left and right with palm faced down; this results in a 90-degree turn left. The 'signal right' is activated by pointing up, causing the car to indicate right. The 'signal left' is activated by pointing down, causing the car to indicate left.

The latency was obtained by connecting both devices to the serial port and running a synchronised clock on the devices. The gesture controller prints the loop processing time and the exact moment a gesture is detected in milliseconds. The car then prints the exact moment a gesture is classified to its desired movement, also in milliseconds. There is also a gap between the times, which is caused by data transmission via wifi.

Latency = Gesture processing time (ms) + Car classifier processing time(ms) + data transmission time (ms)

$$Uncertainty = \frac{Max - min (Range)}{2}$$

Uncertainty was also found using the standard deviation to give a better representation of the spread.

Gesture	Latency Uncertainty (ms)	Angle Uncertainty (ms)
Forward	±6.84	
Reverse	±5.84	
Left	±5.59	
Right	±5.91	
Stop	±3.24	
Wave	±9.06	
Tsunami	±9.06	±3.16
Comsi	±9.21	±2.94
Signal Right	±5.08	
Signal Left	±5.04	

Table 8 – Uncertainties using standard deviation

$$s = \sqrt{\frac{\sum(X - \bar{x})^2}{n - 1}}$$

X - The Value in the data distribution

$\bar{x}$ - The Sample Mean

n - Total Number of Observations

5.2 Auto Calibration System Testing

5.2.1 Sensor Drift Test

This section tests the control system with and without feedback/offset correction. The First two graphs show the angles with closed-loop drift correction, and the last two graphs show open-loop drift correction. The iteration number can be converted to ms, as 1 loop iteration is 10ms. There is an initial test done at 1 second with the device left stationary. The controller is utilised for 3 minutes (performing gestures) to build up sensor drift. 100 loop iterations = 1 second and 18000 loop iterations = 3 minutes.

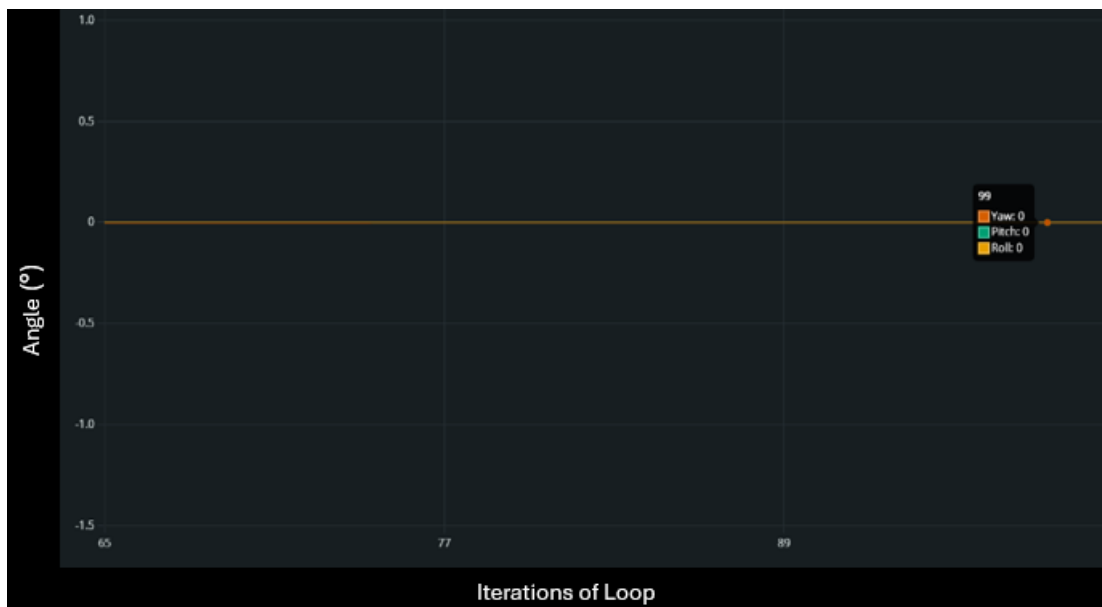


Figure 30. Angle values (Closed loop test) (1 second)

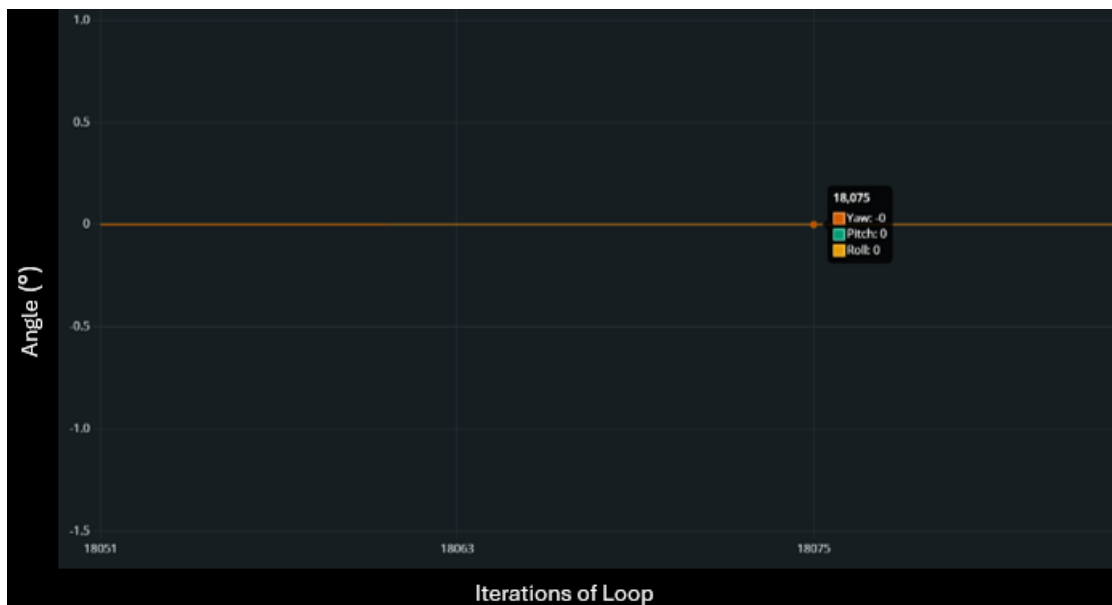


Figure 31. Angle values (Closed loop test) (3 minutes)



Figure 32. Angle values (open loop test) (1 second)

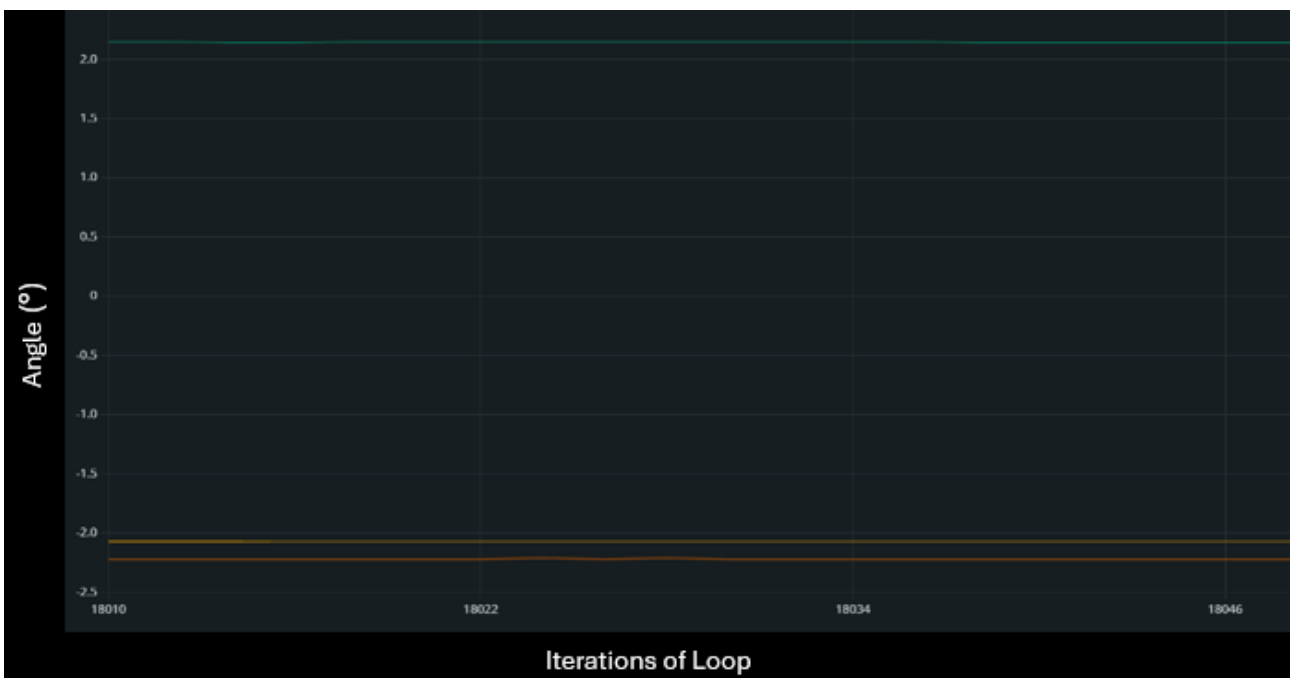


Figure 33. Angle values (open loop test) (3 minutes)

5.2.2 Calibrating System - Error Correction response

This section shows how the Calibration system responds to given drift values. There is an approximate 2.5-second delay between each calibration (the reason for this is explained in section 4.3.1). The tests were done, leaving the device stationary to assess the overall performance of the system.

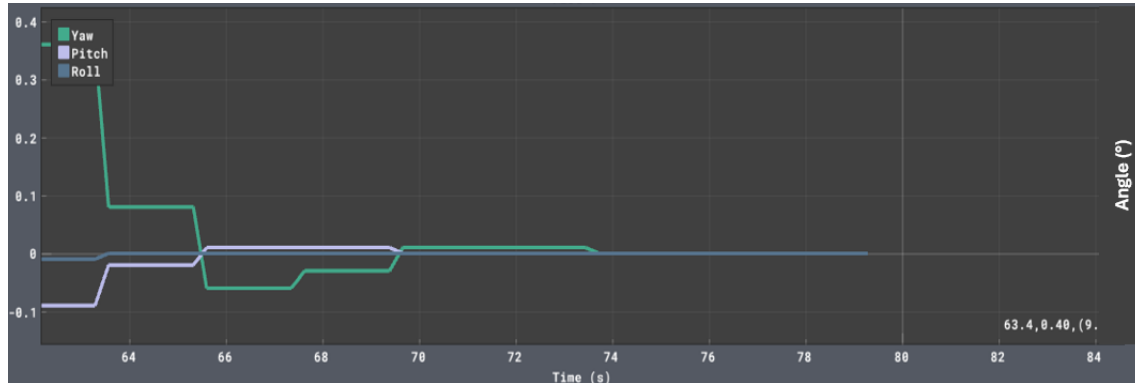


Figure 34. Error correction Test 1 (**Initial Drift**; Yaw: 0.36, Pitch: -0.09 and Roll: -0.01)

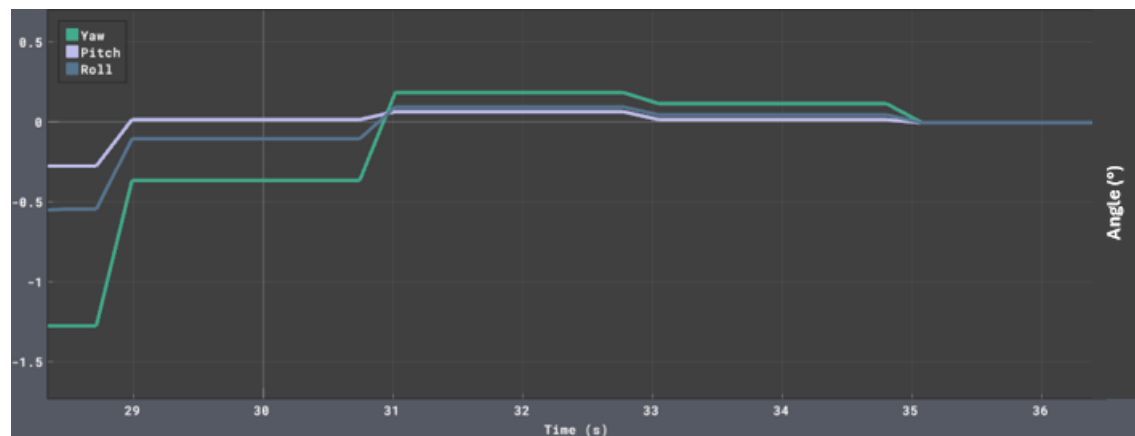


Figure 35. Error correction Test 2 (**Initial Drift**; Yaw: -1.25, Pitch: -0.25 and Roll: -0.55)

Chapter 6: Evaluation

6.1 Performance Assessment

The gesture controller had an **average accuracy of 98.3%**, indicating high overall performance and gesture recognition reliability. Most gestures were classified correctly with near-perfect accuracy. The only significant dip (89%) occurred in tests for the gesture 'Comsi' due to the gesture closely resembling the "wave" gesture, leading to temporary misclassification. This highlights a potential area for improvement in distinguishing between similar gestures. Despite this, the system successfully recognised the correct gesture in 90% of the trials with over 95% accuracy, demonstrating strong robustness and suitability for practical use in controlled environments.

The **average latency** was approximately **45.8 ms**, with most gestures falling between **30 ms and 35 ms**, indicating fast and consistent recognition. However, three gestures that involved the machine learning classifier exhibited higher latency, reaching **105.7 ms**. The classifier processed these gestures, which inherently introduced additional computation time compared to the more straightforward gestures.

The **uncertainties** ranged from **± 3.24 ms to ± 9.06 ms (Standard deviation method)**, reflecting some variability in response time, particularly for gestures involving more complex processing. One observation found during testing was that as the ESP32 microcontroller on the gesture controller heats up due to an extended period of usage, the latency values start to increase and fluctuate.

The angle measurements for the car's 90-degree turns were evaluated to assess the precision and consistency of the system's response. The recorded angles were **90°** and **90.48°**, with associated uncertainties of **$\pm 2.94^\circ$** and **$\pm 3.16^\circ$** , respectively. These results indicate that the system can consistently execute 90-degree turns, with only minimal deviation. The small uncertainties suggest that the system is precise in determining the correct angle.

6.2 Auto Calibration Test Anyalsis

The sensor drift test (5.2.1) showed that without the closed-loop calibration, there is an apparent buildup of drift. In the Open-loop test, after 3 minutes of use, there was an increase in Yaw by +2.4 degrees and decrease in pitch by -1.6 degrees and a decrease in Roll by -0.7 degrees. When the offset correction is implemented (Closed loop), there is no drift apparent.

The error correction responses shown in section 5.2.2 show the device correcting drift values when stationary. The first test took 5 iterations of the Fast calibration to achieve Nulled values, and the second test took 4 iterations. The overall time to calibrate to 0 was 10-12 seconds; these values were high but were a result of the 2.5-second delay introduced. It is also apparent from the graphs that there is the greatest correction in the first attempt at correction, in test 1 Pitch went from 0.35 to 0.09, Pitch from -0.09 to -0.02 and Roll from -0.005 to 0. This is also apparent from test 2. This is beneficial as transitions from gestures such as Forward/Reverse and Left/Right, the device passes through the stationary point briefly, allowing for 1 iteration of the Fast calibration.

An overview of findings: Closed-loop calibration using mean bias correction significantly enhances IMU stability by nullifying drift during stationary phases. The system is capable of self-correction in under 12 seconds, with the largest error corrections typically occurring in the first iteration. This efficiency supports robust real-time performance in gesture-based systems, ensuring that drift does not compromise accuracy during operation.

6.3 Comparisons to Existing Systems

The gesture controller developed in this project achieved an **average accuracy of 98.3%** and an **average latency of approximately 45.8 ms**, with classifier-based gestures peaking at around **105.7 ms**. Compared to other systems in section 2.4, this performance is highly competitive. The device outperforms the existing system, which also has an accuracy of 98%, by having a peak latency of 105.7ms compared to the 330ms of the existing system.

The presented gesture controller maintains high accuracy with relatively low latency, especially considering it handles both simple and machine-learning-based gestures. This balance between precision and responsiveness positions the system as a strong candidate for real-time applications such as robotic control or interactive systems.

Chapter 7: Conclusion

7.1 Achievements

The project successfully met its core objectives by developing a fully functional gesture-controlled RC car equipped with an intelligent recognition system. An auto-calibration mechanism was implemented. Machine learning models were integrated to enhance gesture recognition accuracy and adaptability, resulting in an impressive average accuracy of 98.3%. The system demonstrated solid real-time performance with an average latency of 45.8 ms. However, the original aim of achieving a 30 ms latency target was not fully met, primarily due to the additional processing time required by the machine learning classifier. Despite this, extensive testing confirmed the system's reliability, and comparative analysis showed that it performs competitively or matches conventional gesture-controlled systems in both accuracy and responsiveness.

At the start of this project, I had little to no knowledge of machine learning and ESP32 microcontrollers. Through the development process, I gained a solid understanding of how to implement and train machine learning models for gesture recognition and how to integrate them with embedded systems. I also learned how to work with the ESP32 platform, including handling sensor data, optimising performance, and managing real-time responses. This project greatly expanded my technical skills and gave me valuable hands-on experience in building intelligent, responsive systems.

7.2 Challenges

Throughout the project, several challenges were encountered. One major issue was distinguishing between similar gestures, which sometimes led to misclassifications. Implementing the machine learning model on the ESP32 posed difficulties due to its limited processing power and memory. Additionally, maintaining consistent latency under 30 ms proved challenging, particularly for gestures requiring classification. Designing a suitable and user-friendly calibration system also took considerable iteration to ensure adaptability and accuracy. Hardware-related issues, such as the devices heating up during extended use, affected performance and reliability, requiring careful power management and testing.

Time management was another significant challenge, especially as this project was undertaken in the final year, balancing coursework, exams, and project deadlines. Despite these obstacles, each challenge provided valuable learning experiences and opportunities for improvement.

7.3 Future Improvements

Several improvements can be made to enhance the gesture control system. First, refining the machine learning model to reduce latency below 30 ms and improve gesture classification accuracy is a key goal. Expanding the dataset and incorporating more diverse gestures would help eliminate misclassifications, particularly for similar movements. Optimising the system's hardware to manage heat better and reduce processing delays is another important area for improvement.

Enhancing the calibration system to make it more responsive and adaptable would further improve the system's versatility. Additionally, exploring more efficient power management solutions could extend the system's operational time and reliability and reduce overall operating temperatures.

Furthermore, utilise a more powerful controller to achieve the 30ms latency. This could include using a PC to perform the machine learning classification. Where the gesture controller can communicate with the car and the PC through wifi. This could reduce the latency significantly as the microcontrollers are significantly outperformed by desktop processors. Lastly test on the range of the communication could be done.

7.4 Final Conclusions

This project successfully developed a gesture-controlled RC car using machine learning and auto-calibration, achieving high accuracy and responsive performance. Despite challenges such as distinguishing similar gestures, hardware limitations, and maintaining low latency, the system proved effective in real-world scenarios. The average accuracy of 98.3% and a latency of 45.8 ms demonstrate the system's potential, though the initial goal of achieving sub-30 ms latency was not fully realised. With further optimisation, especially in machine learning model efficiency, calibration, and hardware management, the system could be significantly improved. Overall, the project has provided valuable hands-on experience in embedded systems, machine learning, and real-time performance evaluation, laying a strong foundation for future enhancements.

References

- [1] Yasen M, Jusoh S. A systematic review on hand gesture recognition techniques, challenges and applications. *PeerJ Comput Sci.* 2019 Sep 16;5:e218. doi: [10.7717/peerj-cs.218](https://doi.org/10.7717/peerj-cs.218)
- [2] “What is Gesture Recognition? Use Cases, Technologies and Algorithms,” *GeeksforGeeks*, Mar. 29, 2024. <https://www.geeksforgeeks.org/what-is-gesture-recognition/>
- [3] J. Qi, L. Ma, Z. Cui, et al., “Computer vision-based hand gesture recognition for human-robot interaction: a review,” *Complex Intell. Syst.*, vol. 10, pp. 1581–1606, 2024. doi: [10.1007/s40747-023-01173-6](https://doi.org/10.1007/s40747-023-01173-6).
- [4] B. K. Chakraborty, D. Sarma, M. K. Bhuyan, and K. F. MacDorman, “Review of constraints on vision-based gesture recognition for human–computer interaction,” *IET Comput. Vis.*, vol. 12, no. 1, pp. 3–15, 2018. doi: [10.1049/iet-cvi.2017.0052](https://doi.org/10.1049/iet-cvi.2017.0052).
- [5] R. Tchantchane, H. Zhou, S. Zhang, and G. Alici, “A review of hand gesture recognition systems based on noninvasive wearable sensors,” *Adv. Intell. Syst.*, vol. 5, no. 10, p. 2300207, 2023. doi: [10.1002/aisy.202300207](https://doi.org/10.1002/aisy.202300207).
- [6] K. Nikolopoulou, “What is machine learning? | A beginner’s guide,” *Scribbr*, Aug. 21, 2023. [Online]. Available: <https://www.scribbr.co.uk/using-ai-tools/what-is-machine-learning/>. [Accessed: Nov. 23, 2024].
- [7] GeeksforGeeks, “Vision Transformer (ViT) Architecture,” *GeeksforGeeks*, Oct. 09, 2024. <https://www.geeksforgeeks.org/vision-transformer-vit-architecture/>
- [8] Geeksforgeeks, “Convolutional Neural Network (CNN) in Machine Learning,” *GeeksforGeeks*, Dec. 25, 2020. <https://www.geeksforgeeks.org/convolutional-neural-network-cnn-in-machine-learning/>
- [9] “Top 6 Machine Learning Classification Algorithms,” *GeeksforGeeks*, Feb. 26, 2024. <https://www.geeksforgeeks.org/top-6-machine-learning-algorithms-for-classification/>
- [10] S. Segu, “Understanding and mastering IMU intrinsic calibration,” *Medium*, Aug. 09, 2024. [Online]. Available: <https://medium.com/@sugunsegu/understanding-and-mastering-imu-intrinsic-calibration-ab1e4a4dd3ea>. [Accessed: Nov. 23, 2024].
- [11] D. Jain, “Data normalisation in data mining,” *GeeksforGeeks*, Jun. 13, 2019. [Online]. Available: <https://www.geeksforgeeks.org/data-normalization-in-data-mining/>.
- [12] I. Beavers, “The case of the misguided gyro | Analog Devices,” *Analog Devices*, 2024. [Online]. Available: <https://www.analog.com/en/resources/analog-dialogue/raqs/raq-issue-139.html>.
- [13] L. Fan et al., “Smart-data-glove-based gesture recognition for amphibious communication,” *Micromachines*, vol. 14, no. 11, p. 2050, 2023. doi: [10.3390/mi14112050](https://doi.org/10.3390/mi14112050).
- [14] H. A. Neamah, M. A. Khudhair, and M. S. Dhaiban, “Navigating attention-centric: A machine learning approach to EMG-based hand gesture recognition for interactive RC car,” in *Proc. IEEE 21st Int. Power Electron. Motion Control Conf. (PEMC)*, Pilsen, Czech Republic, 2024, pp. 1–6. doi: 10.1109/PEMC61721.2024.10726353.

[15] A. Filipowska et al., “Machine learning-based gesture recognition glove: Design and implementation,” *Sensors (Basel)*, vol. 24, no. 18, p. 6157, 2024. doi: [10.3390/s24186157](https://doi.org/10.3390/s24186157).

[16] D. W. O. Antillon, C. R. Walker, S. Rosset, and I. A. Anderson, “Glove-based hand gesture recognition for diver communication,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 9874–9886, Dec. 2023. doi: 10.1109/TNNLS.2022.3161682.

[17] R. Arora, “I2C Bus Pullup Resistor Calculation Application Report I2C Bus Pullup Resistor Calculation,” 2015. Accessed: Apr. 28, 2025. [Online]. Available: <https://www.ti.com/lit/an/slva689/slva689.pdf>

Appendix A – Source code

The source code for this project, including the hardware and software implementations, is available on GitHub at <https://github.com/raakin34/Gesture-Controller-Project.git>.

Appendix B - Initial Mean Bias Calibration

```
for (int i = 0; i < 500; i++) {  
    if (bno08x.getSensorEvent(&sensorValue)) {  
        switch (sensorValue.sensorId) {  
            case SH2_ARVR_STABILIZED_RV:  
                quaternionToEulerRV(&sensorValue.un.arvrStabilizedRV, &ypr, true);  
                break;  
            case SH2_GYRO_INTEGRATED_RV:  
                quaternionToEulerGI(&sensorValue.un.gyroIntegratedRV, &ypr, true);  
                break;  
        }  
        // Collect offsets properly  
        OffYaw += ypr.yaw;  
        OffPitch += ypr.pitch;  
        OffRoll += ypr.roll;  
    }  
    delay(5); // Wait for new data  
}  
  
// Compute the average  
OffYaw /= 500;  
OffPitch /= 500;  
OffRoll /= 500;  
}
```

Appendix C - Rapid Mean Bias Calibration

```

for (int i = 0; i < 5; i++) {

    if (bno08x.getSensorEvent(&sensorValue)) {

        switch (sensorValue.sensorId) {

            case SH2_ARVR_STABILIZED_RV:
                quaternionToEulerRV(&sensorValue.un.arvrStabilizedRV, &ypr, true);
                break;

            case SH2_GYRO_INTEGRATED_RV:
                quaternionToEulerGI(&sensorValue.un.gyroIntegratedRV, &ypr, true);
                break;
        }

        // Collect offsets properly
        RapidYaw += ypr.yaw - OffYaw;
        RapidPitch += ypr.pitch - OffPitch;
        RapidRoll += ypr.roll - OffRoll;
    }
    delay(5); // Wait for new data
}

RapidYaw /= 5;
RapidPitch /= 5;
RapidRoll /= 5;
// Compute the average

OffYaw = OffYaw + RapidYaw;
OffPitch = OffPitch + RapidPitch;
OffRoll = OffRoll + RapidRoll;
//Adds new existing offsets to offset variables

```

Appendix D - Gesture classification evaluation

```
if(result.classification[0].value > 0.75){
    comsic++;

    if(tsuc < 3 && wavec < 3 && comsic < 2){
        return;}

    } else{
        comsic = 0;
    }

if(result.classification[2].value > 0.70){
    wavec++;

    if(wavec < 3 && tsuc < 3 && comsic < 2){
        return;}

    } else{
        wavec = 0;
    }

if(result.classification[1].value > 0.75){
    tsuc++;

    if(tsuc < 3 && wavec < 3 && comsic < 2){
        return;}

    } else{
        tsuc = 0;
    }
}
```

Appendix E - RC Car Direction gesture classification

```
if(tsuc < 3 && wavec < 3 && comsic < 2){  
  
    if(ypr.pitch - OffPitch < -15){  
        Serial.println("left");  
        send_value = 40; //Left  
        staC = 0;  
        Speed();  
  
    } else if (ypr.pitch - OffPitch > 15){  
        Serial.println("right");  
        send_value = 30; //Right  
        staC = 0;  
        Speed();  
  
    } else if (ypr.roll - OffRoll > 15){  
        Serial.println("forward");  
        send_value = 10; //Forward  
        staC = 0;  
        Speed();  
  
    } else if (ypr.roll - OffRoll < -15){  
        Serial.println("reverse");  
        send_value = 20; //Reverse  
        staC = 0;  
        Speed();  
  
    }else{  
        Serial.println("stationary");  
        send_value = 50; //Stationary  
        if(staC == 50 && CalibC == 0){  
            rapidcom();  
            CalibC = 1;  
            staC = 0;  
        }  
    }  
}
```

Appendix F - Non-car directional gestures classification

```
if(tsuc > 2){  
    Serial.println("Tsunami");  
    send_value = 70; //Forward  
    delay(50);  
}  
  
if(wavec > 2){  
    Serial.println("wave");  
    send_value = 60; //Forward  
    delay(50);  
}  
  
if(comsic > 1){  
    Serial.println("comsi");  
    send_value = 80; //Forward  
    delay(50);  
}
```


Appendix G – Remote Control Car Code

```
//-----Load libraries
#include <esp_now.h>
#include <WiFi.h>
//-----

#define IN1 5
#define IN2 18
#define IN3 19
#define IN4 21

#define ENA 2
#define ENB 4

#define IN5 26
#define IN6 25
#define IN7 33
#define IN8 32

#define ENC 14
#define END 27

#define LED_PIN1 22
#define LED_PIN2 23

//-----Define variables to store
incoming readings
int receive_value = 0;
//-----

typedef struct struct_message {
    int value;
} struct_message;

struct_message receive_Data; //--> Create a struct_message to receive
data.
//-----

// Callback when data is received

void OnDataRecv(const esp_now_recv_info* recv_info, const uint8_t*
incomingData, int len){
    memcpy(&receive_Data, incomingData, sizeof(receive_Data));
    Serial.println();
    Serial.println("<<<<< Receive Data:");
    Serial.print("Bytes received: ");
    Serial.println(len);
    receive_value = receive_Data.value;
    Serial.println("Receive Data: ");
    Serial.println(receive_value);
    Serial.println("<<<<<");
}
```

```

void setup() {

    Serial.begin(115200);

    WiFi.mode(WIFI_STA); //--> Set device as a Wi-Fi Station

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENA, OUTPUT);
    pinMode(ENB, OUTPUT);

    pinMode(IN5, OUTPUT);
    pinMode(IN6, OUTPUT);
    pinMode(IN7, OUTPUT);
    pinMode(IN8, OUTPUT);
    pinMode(ENC, OUTPUT);
    pinMode(END, OUTPUT);

    pinMode(LED_PIN1, OUTPUT);
    pinMode(LED_PIN2, OUTPUT);

    //-----Init ESP-NOW
    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return;
    }
    //-----
    //--> Register for a callback function that will be called when data
is received
    esp_now_register_recv_cb(OnDataRecv);
}

void loop() {
    // put your main code here, to run repeatedly:

    int gesture = receive_value / 10; // Removes last digit → 43 / 10 =
4
    int speed = receive_value % 10; // Last digit → 3|

    if (gesture == 0) { //Stop

        digitalWrite(IN1, LOW);
        digitalWrite(IN2, LOW);

        digitalWrite(IN3, LOW);
        digitalWrite(IN4, LOW);

        digitalWrite(IN5, LOW);
        digitalWrite(IN6, LOW);

        digitalWrite(IN7, LOW);
        digitalWrite(IN8, LOW);
    }
}

```

```
if (gesture == 1) { //Forward

analogWrite(ENA, 100+(speed*17));
analogWrite(ENB, 100+(speed*17));
analogWrite(ENC, 100+(speed*17));
analogWrite(END, 100+(speed*17));

digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);

digitalWrite(IN5, HIGH);
digitalWrite(IN6, LOW);
digitalWrite(IN7, HIGH);
digitalWrite(IN8, LOW);
}

if (gesture == 2) { //Reverse

analogWrite(ENA, 100+(speed*17));
analogWrite(ENB, 100+(speed*17));
analogWrite(ENC, 100+(speed*17));
analogWrite(END, 100+(speed*17));

digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);

digitalWrite(IN5, LOW);
digitalWrite(IN6, HIGH);
digitalWrite(IN7, LOW);
digitalWrite(IN8, HIGH);
}

if (gesture == 3) { //Right

analogWrite(ENA, 100+(speed*17));
analogWrite(ENB, 100+(speed*17));
analogWrite(ENC, 100+(speed*17));
analogWrite(END, 100+(speed*17));

digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, HIGH);

digitalWrite(IN5, LOW);
digitalWrite(IN6, HIGH);
digitalWrite(IN7, HIGH);
digitalWrite(IN8, LOW);
}
```

```
if (gesture == 4) { //Left

analogWrite(ENA, 100+(speed*17));
analogWrite(ENB, 100+(speed*17));
analogWrite(ENC, 100+(speed*17));
analogWrite(END, 100+(speed*17));

digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);

digitalWrite(IN5, HIGH);
digitalWrite(IN6, LOW);
digitalWrite(IN7, LOW);
digitalWrite(IN8, HIGH);
}

if (gesture == 5) { //Stationary
digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);

digitalWrite(IN5, LOW);
digitalWrite(IN6, LOW);
digitalWrite(IN7, LOW);
digitalWrite(IN8, LOW);
}

if (gesture == 6) { //Flash
digitalWrite(LED_PIN1, HIGH);
digitalWrite(LED_PIN2, HIGH);
delay(500);
digitalWrite(LED_PIN1, LOW);
digitalWrite(LED_PIN2, LOW);
delay(500);
}

if (gesture == 7) { // 90 degree turn left
analogWrite(ENA, 255);
analogWrite(ENB, 255);
analogWrite(ENC, 255);
analogWrite(END, 255);

digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);

digitalWrite(IN5, HIGH);
digitalWrite(IN6, LOW);
digitalWrite(IN7, LOW);
digitalWrite(IN8, HIGH);

delay(405);

analogWrite(ENA, 0);
analogWrite(ENB, 0);
analogWrite(ENC, 0);
analogWrite(END, 0);
}
```

```
    if (gesture == 8) { 90 degree trun right
    analogWrite(ENA, 255);
    analogWrite(ENB, 255);
    analogWrite(ENC, 255);
    analogWrite(END, 255);

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);

    digitalWrite(IN5, LOW);
    digitalWrite(IN6, HIGH);
    digitalWrite(IN7, HIGH);
    digitalWrite(IN8, LOW);

    delay(385);

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    analogWrite(ENC, 0);
    analogWrite(END, 0);
    }

    if (gesture == 9) { indicate right

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    analogWrite(ENC, 0);
    analogWrite(END, 0);

    digitalWrite(LED_PIN2, HIGH);
    delay(500);

    digitalWrite(LED_PIN2, LOW);
    delay(500);
    }

    if (gesture == 10) { indicate left

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    analogWrite(ENC, 0);
    analogWrite(END, 0);

    digitalWrite(LED_PIN1, HIGH);

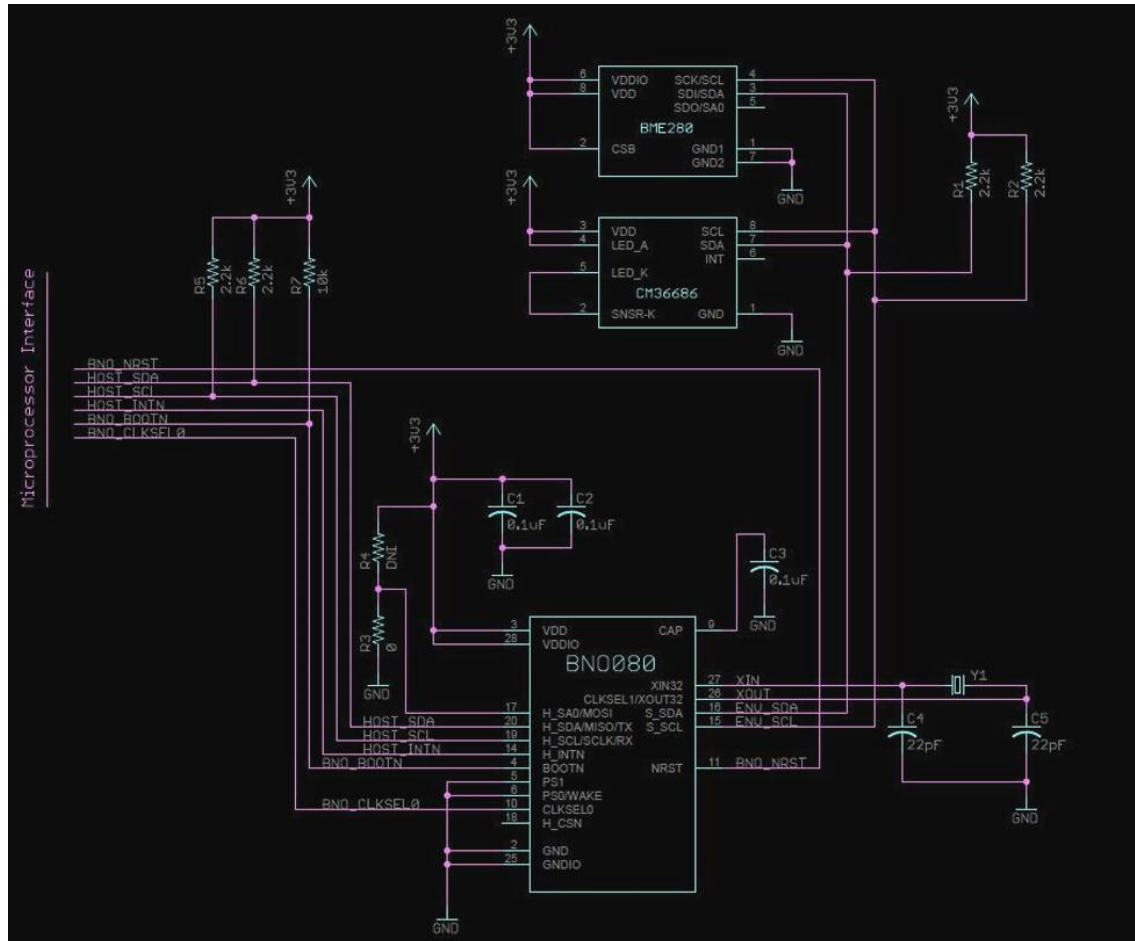
    delay(500);
    digitalWrite(LED_PIN1, LOW);
    delay(500);
    }

    delay(10);
}
```

Appendix H - Speed evaluation code

```
void Speed() {  
  // Handle speed calculation using a switch  
  int speed;  
  
  switch (send_value) {  
    case 40: // Left  
      speed = (abs(ypr.pitch - OffPitch) - 15) / 6;  
      break;  
  
    case 30: // Right  
      speed = (abs(ypr.pitch - OffPitch) - 15) / 6;  
      break;  
  
    case 10: // Forward  
      speed = (abs(ypr.roll - OffRoll) - 15) / 6;  
      break;  
  
    case 20: // Reverse  
      speed = (abs(ypr.roll - OffRoll) - 15) / 6;  
      break;  
  }  
  
  // Cap speed to 9 if over 64degreea  
  if (speed > 9) {  
    speed = 9;  
  }  
  send_value = send_value + speed;  
}
```

Appendix I – BNO08X Circuit Diagram [17]



Abbreviations list.

RC: Remote Control

VBGR: Vision-Based Gesture Recognition

SBGR: Sensor-Based Gesture Recognition

IMU: Inertial measurement units

FMG: Force myography

CNN – Convolutional neural networks

ViTs - Vision Transformer Technique

SVM – Support vector machines

NN – Neural networks

TIMEPLAN			
Semester 1			Semester 2
Week 1		Week 1	Start creating auto-calibration algorithms.
Week 2		Week 2	
Week 3		Week 3	
Week 4	Project Definition and Plan Submission	Week 4	Machine learning for advanced gesture recognition
Week 5	Research and find suitable components identifying dimensions to create a model for The RC car	Week 5	
Week 6	Sourcing components for the RC car and gesture controller	Week 6	
Week 7	CAD design for the RC car/3d print Chassis	Week 7	
Week 8	Start the creation of the RC car	Week 8	
Week 9		Week 9	
Week 10	Finish RC car creation Interim Report Submission- 5%- Part 1	Week10	

Week 11	Start creation of the gesture controller	Week 11	
Week 12	Finish creation of the gesture controller	Week12	Final tests of the system.

Potential Risks to the project

ID	Risk	Likelihood	Impact	Actions
1	Short circuit	Low – circuit will be thoroughly tested before being made. And batteries have shorting protection	Components or batteries may be heavily damaged	Replacing components
2	Buring from Soldering	Low – Minimal soldering is to be done with care and wearing correcting safety equipment such as goggles and gloves	Pain and discomfort from burns	Training and Awareness and taking time when soldering.
3	Component Failures	Low– Components will be tested and secured to the device	Poorly secured parts can detach during operation, posing safety risks to bystanders	Design and assembly checks, reassure the design is suitable and safe